

ADDIS ABABA UNIVERSITY

COLLEGE OF NATURAL AND COMPUTATIONA SCIENCE

GIZEGO REAL-TIME BUS TRACKING SIMULATION

Members

1. Sofanit Mesfin (UGR/2417/15)

2. Samrawit Haileeyesus(UGR/5530/15)

3. Rediet Yalelet(UGR/7139/15)

4. Tesfa Tilahun(UGR/0647/15)

Date: 24/01/2026

Supervisor: Mr.Leykun Birhanu.

GIZEGO REAL-TIME BUS TRACKING SIMULATION

This project documentation submitted in partial fulfillment of the requirements of Degree of Bachelor of Science in Computer Science.

Project Advised by: Leykun Birhan

Name and Signature of Members of the examining board:

Name	Title	Signature
1.		
2.		
3.		

DECLARATION

This is to declare that this project work, which is done under the supervision of Mr. Leykun B. and having the title GizeGo is the sole contribution of Rediet Yalelet, Samrawit Hailaeyesus, Sofanit Mesfin, Tesfa Tilahun.

No part of the project work has been reproduced illegally (copy and paste) which can be considered as Plagiarism. All referenced parts have been used to argue the idea and have been cited properly. We will be responsible and liable for any consequence if violation of this declaration is proven.

Date: _____

Group Members:

Full Name Signature

1. Rediet Yalelet
2. Samrawit Hailaeyesus
3. Sofanit Mesfin
4. Tesfa Tilahun

CERTIFICATE

I certify that this BSc final project report entitled GizeGo by:

1. Rediet Yalelet
2. Samrawit Hailaeyesus
3. Sofanit Mesfin
4. Tesfa Tilahun

is approved by me for submission. I certify further that, to the best of my knowledge, The report represents work carried out by the students.

_____ _____

Date Name and Signature of Supervisor

ACKNOWLEDGEMENT

First and foremost, we would like to thank Almighty God for granting us the strength, patience, and guidance throughout this journey. We extend our sincere appreciation to our adviser, Mr. Leykun Birhanu for his steadfast support and guidance throughout our final project. His invaluable feedback and constructive criticism has significantly contributed to the refinement of our work.

We also sincerely thank our families for their continuous encouragement, understanding, and for standing by us with both moral and financial support as we worked toward achieving our goals.

ABSTRACT

Public transportation in Addis Ababa is a critical service for urban mobility, yet it is affected by challenges such as unpredictable schedules, traffic congestion, and the lack of accessible real-time travel information. These limitations result in increased waiting times, inefficient route planning, and reduced commuter satisfaction. This project presents GizeGo, a simulation-based real-time public transportation tracking and route optimization system developed as an academic prototype. The system uses simulated GPS data to represent the movement of virtual buses along predefined routes and integrates OpenStreetMap (OSM) for map visualization and routing. Through a web and mobile-based interface with bilingual (Amharic/English) support, GizeGo allows users to view simulated bus locations, routes, and estimated times of arrival (ETAs) in real time.

The GizeGo system employs the A* pathfinding algorithm combined with simulated traffic conditions to optimize routes and dynamically update ETA predictions. The system architecture follows a layered design consisting of presentation, business logic, and data access layers to ensure modularity, scalability, and maintainability. Real-time updates are delivered using WebSocket communication within the simulated environment, while analytics components generate performance insights based on historical simulation data. Although the system does not rely on physical GPS devices or live transportation data, it effectively demonstrates the feasibility of a low-cost, scalable public transportation tracking solution and provides a strong foundation for future enhancement and real-world deployment.

Content

page

GIZEGO REAL-TIME BUS TRACKING SIMULATION i
DECLARATION ii
CERTIFICATE iii
ACKNOWLEDGEMENT iv
ABSTRACT v
List of Tables x
List of Figures xi
List of Acronyms xii
CHAPTER ONE 1
PROJECT PROPOSAL 1
1.1 Introduction 1
1.2 Statement of the Problem 3
1.3 Objective 4
1.3.1 General Objective 4
1.3.2 Specific Objective 4
1.4 Literature Review 5
1.4.1 Real-Time Tracking Systems in Public Transport 5
1.4.2 Open-Source Geospatial Tools for Mapping 5
1.4.3 Routing and Pathfinding Algorithms 6
1.4.4 Technology Stacks for Real-Time Mobile Applications 6
1.4.5 Identification of the Research Gap 6
1.5 Scope and Limitation 7
1.5.1 System Development: 7
1.5.2 Features Covered: 7
1.5.3 Technology and Integration: 7
1.5.4 Limitations 8
1.6 System Development Methodology 8
1.6.1 Approach 8
1.6.2 Tools and Technologies 9

1.6.3 Data Collection 9
1.7 Significance of the Project 10
1.8 Beneficiaries 11
1.9 Project Timeline 12
1.9.1 Documentation Phase 12
1.9.2 Implementation Phase 13
CHAPTER TWO 15
SYSTEM AND REQUIREMENT ANALYSIS 15
2.1 Introduction 15
2.2 Current System 16
2.2.1 Major Functions of the Current System 16
2.2.2 Problems of the Existing System 17
2.3 Requirement Gathering 18
2.3.1 Requirement Gathering Methodologies 19
2.3.2 Results Found 19
2.4 Proposed System 20
2.4.1 Overview 20
2.4.2 Functional Requirements 20
2.4.3 Non-Functional Requirements 22
2.5 Assumptions and Project Limitations 23
2.5.1 Assumptions 23
2.5.2 Project Limitations 24
2.6 System model 25
2.6.1 Scenario 25
2.6.1.1 As is scenario 25
2.6.1.2 Visionary Scenario 26
2.6.1.3 Evaluation Scenario 26
2.6.1.4 Training Scenario 27
2.6.2 Use case model 28
2.6.2.1 Use case diagram 28
2.6.2.2 Description of use case model 30
2.6.3 Dynamic model 43
2.6.3.1 Sequence diagram 43
2.6.3.2 Activity diagram 56
CHAPTER THREE 62
SYSTEM DESIGN 62

3.1 Introduction 62
3.1.1 Purpose of the System Design 62
3.2 Proposed Software Architecture 64
3.2.1 Architectural Overview 64
3.2.2 Subsystem Decomposition 65
3.2.3 Hardware / Software Mapping 67
3.2.4 Persistent Data Management 69
3.2.5 Access Control and Security 76
3.3 Detailed Class Diagram 77
3.3.1 Conceptual Class Diagram 77
3.3.2 Description of Main Classes 78
3.4 Package Diagram 82
3.4.1 Overview of Package Organization 82
3.4.2 Presentation Layer 82
3.4.2.1 Mobile Application Package 82
3.4.2.2 Web Dashboard Package 83
3.4.3 Business Logic Layer 83
3.4.3.1 Route Management Package 83
3.4.3.2 Tracking Service Package 84
3.4.3.3 ETA Computation Package 84
3.4.3.4 Notification Service Package 85
3.4.3.5 Authentication and Authorization Package 85
3.4.3.6 Analytics Package 85
3.4.4 Data Access Layer 86
3.4.5 Integration and Utilities Layer 86
3.4.5.1 External Services Package 86
3.4.5.2 Utilities Package 86
3.4.6 Package Dependencies 87

3.5 UI Design 89
3.6 Design Constraints and Assumptions 91
3.6.1 Design Assumptions 91
3.6.1.1 User Assumptions 91
3.6.1.2 Data and Mapping Assumptions 92
3.6.1.3 System Operation Assumptions 92
3.6.1.4 Performance Assumptions 93

3.6.1.5 Deployment Assumptions 93
3.6.1.6 Future Integration Assumptions 94
Conclusion 96
Reference 96

List of Tables

Table 2.1 Description for Register User use case 30
Table 2.2 Description for Authenticate User use case Table 31
Table 2.3 Description for Update Profile use case 32
Table 2.4 Description for View Real-Time Map use case 33
Table 2.5 Description for Search Route use case 34
Table 2.6 Description for View Bus ETA use case 35
Table 2.7 Description for Report Incident use case 36
Table 2.8 Description for Start GPS Simulation use case 37
Table 2.9 Description for Manage Routes use case 38
Table 2.10 Description for View System Dashboard use case 39
Table 2.11 Description for Update Bus Location use case 40
Table 2.12 Description for Optimize Route use case 41
Table 2.13 Description for Compute ETA use case 42
Table 2.14 Description for Send Notifications use case 43
Table 3.1 Users table 70
Table 3.2 Roles table 71
Table 3.3 Buses table 71
Table 3.4 Routes table 71
Table 3.5 Stops table 71
Table 3.6 Bus_Routes table (Associates buses with routes) 72
Table 3.7 gps_logs table 72
Table 3.8 Incidents table 73
Table 3.9 Notification table 73
Table 3.10 ETA_predictions 74
Table 3.11 Access control 77

List of Figures

Figure 1.1 Documentation phase Gantt chart 13
Figure 1.2 Implementation phase Gantt chart..... 14
Figure 2.1 Use case diagram 29
Figure 2.2 Sequence diagram for Register User..... 44
Figure 2.3 Sequence diagram for Authenticate User 45
Figure 2.4 Sequence diagram Update Profile 46
Figure 2.5 Sequence diagram for View Real-time map 47
Figure 2.6 Sequence diagram for Search Route..... 48
Figure 2.7 Sequence diagram for View Bus ETA 49
Figure 2.8 Sequence diagram for Report Incident 50
Figure 2.9 Sequence diagram for start GPS Simulation 51
Figure 2.10 Sequence diagram for Mange Route 52
Figure 2.11 Sequence diagram for View System Dashboard 53
Figure 2.12 Sequence diagram for Update Bus Location 54
Figure 2.13 Sequence diagram for Optimize Route..... 55
Figure 2.14 Sequence diagram for Compute ETA..... 55
Figure 2.15 Sequence diagram for Send Notification 56
Figure 2.16 Activity diagram for view real time map……57
Figure 2.17 Activity diagram for Search Route..... 58
Figure 2.18 Activity diagram for view bus ETA 59
Figure 2.19 Activity diagram for Start GPS Simulation 60
Figure 2.20 Activity diagram for Report Incident..... 61
Figure 3.1 Three-Tier Architecture 65
Figure 3.2 component diagram..... 67
Figure 3.3 Deployment diagram 69
Figure 3.4 ER diagram 75
Figure 3.5 Class Diagram..... 82
Figure 3.6 Package Diagram..... 88
Figure 3.7 UI design..... 91

List of Acronyms

1. ITS - Intelligent Transport System
2. ETA - Estimated Time of Arrival
3. GPS - Global Positioning System
4. OSM - OpenStreetMap
5. CSV - Comma-Separated Values
6. JSON - JavaScript Object Notation
7. API - Application Programming Interface
8. CI/CD - Continuous Integration / Continuous Deployment
9. HTTPS - Hypertext Transfer Protocol Secure
10. JWT - JSON web token
11. RBAC - role-based access control

Leave this page blank!!

CHAPTER ONE

PROJECT PROPOSAL

1.1 Introduction

Urban mobility is a fundamental component of social, economic, and infrastructural development in modern cities. As Ethiopia's largest and most densely populated urban center, Addis Ababa relies heavily on its public transportation system to support the movement of millions of residents every day. Buses, minibuses, and mid-sized vehicles operated by Anbessa Bus, Sheger Bus, Velocity buses, and informal minibus associations form the backbone of urban transport. A 2018 case study estimated that Addis Ababa's public transport network accommodates approximately 1.9 million passengers daily, illustrating the system's essential role in maintaining the city's functionality. As urban growth accelerates and demand for mobility continues to rise, the limitations of the existing transport network have become increasingly apparent[5].

The growing need for smarter mobility solutions has brought global attention to Intelligent Transportation Systems (ITS), which integrate GPS, data analytics, wireless communication, and real-time monitoring to improve transit efficiency. In many cities worldwide including Nairobi, Kigali, New Delhi, London, and Singapore real-time bus tracking systems have transformed urban mobility by reducing waiting times, optimizing routes, and enhancing passenger satisfaction. These systems provide dynamic estimated time of arrival (ETA) updates, delay notifications, congestion alerts, and route optimization tools that allow operators to make informed decisions quickly. Such technological advancements demonstrate the potential impact similar systems could have in Addis Ababa[5].

However, the implementation of real-time GPS-based tracking in Ethiopia remains limited. While there have been partial telematics implementations on selected buses, these systems are neither comprehensive nor publicly accessible. The high cost of hardware deployment, the need for continuous maintenance, and infrastructural constraints have slowed adoption. As a result,

There is no fully integrated, citywide platform that allows passengers to monitor bus locations, receive ETA updates, or view real-time service disruptions. This gap presents a clear opportunity for research and prototype development.

This project proposes the design and development of a real-time bus tracking and route optimization system tailored specifically to Addis Ababa's transit ecosystem. The system will be implemented as a simulation-based prototype, using simulated GPS data rather than real hardware. This approach allows our project to demonstrate the capabilities, benefits, and functionality of real-time tracking without incurring the cost and logistical challenges associated with physical GPS implementation. Through this simulation, virtual buses will move along predefined routes in real time, allowing passengers to track them via a mobile application and receive accurate estimated arrival times. Drivers and operators within the simulation will also gain access to route optimization tools that recommend less congested paths, helping demonstrate how such a system could improve fleet efficiency in practice.

In summary, the development of a simulation-based real-time bus tracking and route optimization application represents a timely and necessary step toward improving the quality, reliability, and efficiency of public transportation in Addis Ababa. By demonstrating the feasibility and benefits of such a system, this project provides a foundation for future collaboration with transportation authorities and private operators who may expand the concept into a fully operational citywide solution. Ultimately, the proposed system seeks to transform how passengers interact with public transit, enhance operational efficiency, and contribute to a more sustainable and intelligent urban mobility ecosystem in Ethiopia.

1.2 Statement of the Problem

Despite its critical role in daily transportation, Addis Ababa's public transit system continues to experience several long-standing challenges that hinder its efficiency, reliability, and user satisfaction[5]. These challenges include:

1. Unpredictable Schedules

Many buses and minibuses operate without consistent or transparent timetables. Passengers often wait long periods especially in high-demand locations such as Bole, Piassa, Mexico Square, and Megenagna without knowing when the next bus will arrive. This unpredictability discourages public transit use and creates significant time losses for daily commuters.

2. Severe Traffic Congestion

Addis Ababa's major road corridors frequently experience heavy congestion. Roadblocks caused by peak-hour traffic, accidents, construction zones, and narrow roadways further delay transit operations. Without real-time information, drivers are unable to reroute effectively, resulting in prolonged travel times and inefficient fleet usage.

3. Lack of Real-Time Information for Passengers

Passengers currently have no access to live bus location updates, estimated arrival times, or delay notifications. This absence of information increases frustration, reduces perceived reliability, and often forces commuters to cluster at bus stops unnecessarily, contributing to crowding and disorder.

4. Inefficient and Static Route Planning

Transit operators rely on pre-defined routes and static schedules that do not adapt to changing traffic conditions or unexpected disruptions. This leads to operational inefficiencies such as unbalanced load distribution, excessive fuel consumption, and increased environmental impact.

5. Limited Digital Infrastructure and Integration

Although there have been partial attempts such as limited telematics on some Anbessa buses there is still no widely accessible, centralized tracking system that integrates real-time data for public use. Existing systems are either fragmented, incomplete, or not designed with passenger accessibility in mind.

These problems collectively reduce commuter trust, decrease the attractiveness of public transportation, and contribute to increased use of private vehicles exacerbating traffic congestion and carbon emissions. The absence of a centralized, real-time, user-friendly tracking system also limits operators' ability to make fast, data-driven decisions during unexpected delays, breakdowns, or service disruptions[9].

Given these challenges, there is a clear need for a comprehensive, locally adapted real-time bus tracking and route optimization solution for Addis Ababa. Such a system would provide accurate arrival predictions, route guidance, delay alerts, and improved communication between passengers, drivers, and operators even in a simulated environment. No such unified platform currently exists, creating a significant gap that this project aims to address[5].

1.3 Objective

1.3.1 General Objective

The general objective of this project is to design, develop, and implement a simulation-based prototype of a real-time public transportation tracking and route optimization system for Addis Ababa city.

1.3.2 Specific Objective

Realize the general objective, the following specific tasks and activities will be undertaken.

1. Select and model 5-10 major public transportation routes in Addis Ababa using geospatial data from OpenStreetMap (OSM).
2. Design and implement a simulation engine capable of generating and updating mock GPS data for a fleet of 10-20 virtual buses along the modeled routes.

3. Implement the A* pathfinding algorithm on the backend to perform dynamic route optimization based on simulated distance and traffic conditions.
4. Develop a robust backend server using Node.js and Express.js to manage route data, vehicle states, and facilitate real-time communication.
5. Establish a real-time data connection between the server and clients using Socket.io for live tracking updates.
6. Design and build a responsive front-end Mobile application for commuters using React Native, featuring an interactive map, live bus locations, and estimated arrival times (ETAs).
7. Implement bilingual support (Amharic and English) within the commuter mobile application.
8. Design and develop an operator dashboard that visualizes fleet status, route performance, and analytical data using charts and graphs.

1.4 Literature Review

This section reviews specific research and existing systems that are directly relevant to the "GizeGo" project. The review is organized around key themes to establish a foundation, highlight proven methodologies, and identify the specific gap this project aims to address.

1.4.1 Real-Time Tracking Systems in Public Transport

The implementation of real-time tracking has become a standard for modern public transportation systems. For instance, Moovit crowdsources data to provide live directions and arrival times in over 3,000 cities, demonstrating the global scalability of such solutions . Closer to the context of this project, a study on the potential for a digital bus system in Addis Ababa concluded that a lack of real-time information was a primary source of passenger dissatisfaction, strongly justifying the need for systems[6].

1.4.2 Open-Source Geospatial Tools for Mapping

The reliance on proprietary map data and APIs can be a significant barrier in resource-constrained environments. The combination of OpenStreetMap (OSM) and the Leaflet.js library has emerged as a robust, cost-effective alternative. A relevant case study is the

"Digital Matatus" project in Nairobi, which successfully used OSM to map the city's informal minibus network, creating the first formal digital data for the system. This work proves the viability of open-source tools for mapping complex transit networks in African cities. Furthermore, research by Haklay and Weber establishes that OSM data can be of high enough quality for many professional applications, supporting its use in this project[8].

1.4.3 Routing and Pathfinding Algorithms

The core of any route optimization system is its pathfinding algorithm. While Dijkstra's algorithm is a foundational solution for finding the shortest path, the A* algorithm is often preferred for its efficiency in large graphs like road networks. A comprehensive comparison found that A* consistently outperformed Dijkstra in computational speed for network-based pathfinding, especially when a good heuristic is used. This makes A* a suitable choice for a system requiring rapid route recalculation[4].

1.4.4 Technology Stacks for Real-Time Mobile Applications

The choice of the PERN stack (PostgreSQL, Express.js, React Native, Node.js) with Socket.io is supported by its widespread adoption for building scalable, real-time applications. For example, a study on building a real-time collaborative system demonstrated that Node.js with Socket.io provides low-latency communication essential for live updates, making it ideal for tracking applications. This validates the selected architecture for handling the real-time data flow between the GizeGo simulation engine and the client-facing interface[4].

1.4.5 Identification of the Research Gap

While the studies and systems above provide a strong technological foundation, they also reveal a specific gap. Global platforms like Moovit are not tailored to Addis Ababa's unique mix of formal and informal transport, and research projects like Digital Matatus focused on mapping rather than real-time tracking and optimization for commuters. Therefore, this project aims to synthesize these concepts leveraging open-source mapping, efficient algorithms, and a modern stack to create a tailored, simulation-based

prototype that addresses the real-time information gap in Addis Ababa's public transport system.

1.5 Scope and Limitation

1.5.1 System Development:

The system involves the development of a mobile app platform equipped with simulated GPS integration to enable real-time tracking of buses. The platform will display continuous location updates and make the information easily accessible from mobile devices.

The project will also include the implementation of dedicated interfaces for passengers and operators to meet their distinct needs. The passenger interface will provide real-time bus locations, estimated arrival times, and route information, while the operator dashboard will offer tools for monitoring fleet status and managing routes.

Additionally, the system will incorporate route optimization capabilities that calculate the shortest paths or fastest travel times.

1.5.2 Features Covered:

For passengers, the system provides essential real-time transportation features designed to enhance convenience and reduce waiting uncertainty. These include live bus location tracking on an interactive map, detailed route information such as stops and travel paths, and notification services that alert users about estimated arrival times, delays, or changes in service. Together, these features help passengers plan their trips more efficiently and improve their overall commuting experience.

1.5.3 Technology and Integration:

The system will be developed using React Native for frontend, Node.js for backend processing and PostgreSQL for scalable data storage. These technologies support real-time updates, high data throughput, and flexible handling of GPS information.

The platform integrates GPS APIs to simulate or retrieve bus location data and updates the system continuously. Additionally, the architecture is designed with future extensibility in mind allowing seamless integration with third party services such as

national public transport applications, emergency service systems to further expand system capabilities over time.

1.5.4 Limitations

First, the system does not utilize actual GPS hardware; instead, bus location data is generated through simulated GPS inputs. As a result, the accuracy of movement patterns and real-world behavior may not fully reflect actual transportation conditions. Second, the implementation was restricted to a limited geographic area, which reduces the ability to test system performance across varied road networks, traffic conditions, and larger route volumes. Finally, the project was developed under strict time constraints, which limited the inclusion of additional features, extensive testing, and deeper integration with external transportation services.

1.6 System Development Methodology

1.6.1 Approach

1. Simulation Engine

The system begins with a simulation engine designed to generate realistic bus movement without requiring any physical GPS hardware. Mock GPS points are generated along predefined route polylines extracted from OpenStreetMap data. To simulate real world behavior, the engine applies both random variations and time based traffic factors, such as rush hour slowdown or midday free flow conditions.

2. Routing and Optimization

The routing module utilizes the OSM road network as its base dataset. Using algorithms such as A*, the system computes optimal routes based on two primary criteria: shortest distance and least travel time[8].

Traffic weights generated from simulated congestion levels are applied to influence path selection, allowing the platform to calculate alternate routes when certain roads become "congested" ; this provides a realistic environment to test routing logic under multiple conditions and supports analysis for future route optimization.

3. System Architecture

The platform follows a modern, scalable architecture divided into frontend, backend, real-time communication, and data layers.

A. Frontend:

Provides an interactive map-based interface where passengers and operators can view real-time bus positions, routes, and updates. React Native is used for map visualization due to its lightweight nature and compatibility with OSM.

B. Backend:

Handles request processing, routing algorithms, data management, and simulation logic. It also exposes REST APIs for both commuter applications and operator dashboards.

C. Real-Time Communication:

Ensure continuous updates of vehicle positions, enabling smooth and immediate location refresh every few seconds across all client devices.

D. Database:

Stores route definitions, simulated vehicle states, historical tracking data, and user related information. Its flexible structure supports rapid read/write operations required for real-time systems.

E. Analytics Engine:

Aggregates tracking data to generate insights for operators, such as average travel times, peak hours, route performance, and fleet utilization.

1.6.2 Tools and Technologies

Frontend: React Native,
Backend: Node.js, Express.js,
Real-Time: Socket.io,
Database: PostgreSQL
Version Control: Git and GitHub,
Testing: Postman,

1.6.3 Data Collection

No physical GPS devices are required for this project since all bus movement data is produced through controlled software simulation.

Mock GPS data is generated using route polylines obtained from OpenStreetMap. These polylines represent actual road layouts, enabling the simulated buses to follow real-world paths and behave as though they are traveling through actual city streets.

To enhance realism, the simulation applies varying vehicle speeds that may change randomly or follow time-based patterns, such as morning rush hour or late-night reduced traffic. This helps create diverse and dynamic movement behaviors across all simulated buses.

Additionally, publicly available OSM datasets including road networks, bus routes, and assumed traffic conditions are used to construct the routing engine and map interface. These open datasets ensure reliability, accuracy, and reproducibility while keeping the project free from licensing constraints[7].

1.7 Significance of the Project

The GizeGo project is significant because it addresses critical challenges in Addis Ababa's public transportation system by introducing a modern, technology-driven solution for real-time bus tracking and route optimization. By simulating GPS-based tracking and intelligent routing, the project demonstrates how digital systems can improve reliability, efficiency, and user satisfaction in urban mobility.

This project contributes academically by integrating concepts from Intelligent Transportation Systems (ITS), geospatial mapping, real-time communication, and pathfinding algorithms into a unified system. It serves as a practical application of theoretical knowledge in software engineering, data structures, networking, and mobile application development. The use of simulation-based GPS data further provides a low-cost and risk-free environment for testing transportation solutions in resource-constrained settings.

From a practical perspective, the system highlights how real-time information can reduce passenger waiting time, minimize uncertainty, and improve decision-making for both commuters and operators. The route optimization component demonstrates how traffic-aware routing can reduce congestion, fuel consumption, and operational inefficiencies.

Additionally, the bilingual support (Amharic and English) enhances accessibility and inclusivity for a broader range of users.

Overall, the project serves as a proof-of-concept that can inform future real-world implementations by transportation authorities or private operators. It lays a foundation for smarter urban mobility solutions in Ethiopia and contributes toward sustainable, efficient, and data-driven public transportation systems.

1.8 Beneficiaries

The GizeGo system benefits multiple stakeholders involved in public transportation, directly and indirectly, as outlined below:

1. Passengers / Commuters

Passengers are the primary beneficiaries of the system. They gain access to real-time bus location tracking, estimated arrival times (ETAs), route information, and delay notifications. This reduces uncertainty, waiting time, and frustration while improving overall travel planning and commuting experience.

2. Bus Operators and Drivers

Bus operators benefit from improved visibility of fleet movement and route performance. The route optimization and analytics features help drivers and operators make informed decisions during congestion, delays, or route disruptions, leading to better fuel efficiency and service reliability.

3. Transportation Authorities

City transportation agencies and regulators can use insights from the system to analyze traffic patterns, evaluate route efficiency, and plan infrastructure improvements. The system supports data-driven decision-making and policy formulation for urban transport development.

4. Software Developers and Researchers

The project provides a valuable reference for students, developers, and researchers interested in intelligent transportation systems, real-time applications, and simulation-

based system design. It demonstrates how open-source tools and modern technologies can be applied to solve real-world urban problems.

5. The Public and City Environment

By promoting efficient public transport usage, the system indirectly benefits the broader community through reduced traffic congestion, lower carbon emissions, and improved urban mobility. Encouraging public transport use contributes to a more sustainable and environmentally friendly city.

1.9 Project Timeline

The project is organized into two major phases. The first phase focuses on documentation and proposal defense, while the second phase covers system implementation, testing, and final defense.

1.9.1 Documentation Phase

This phase includes all activities required to prepare, review, and defend the project documentation. The schedule covers requirement analysis, literature review, system design, and proposal preparation.

Gantt Chart for Documentation Phase:

The Gantt chart illustrating the 40-day documentation schedule:

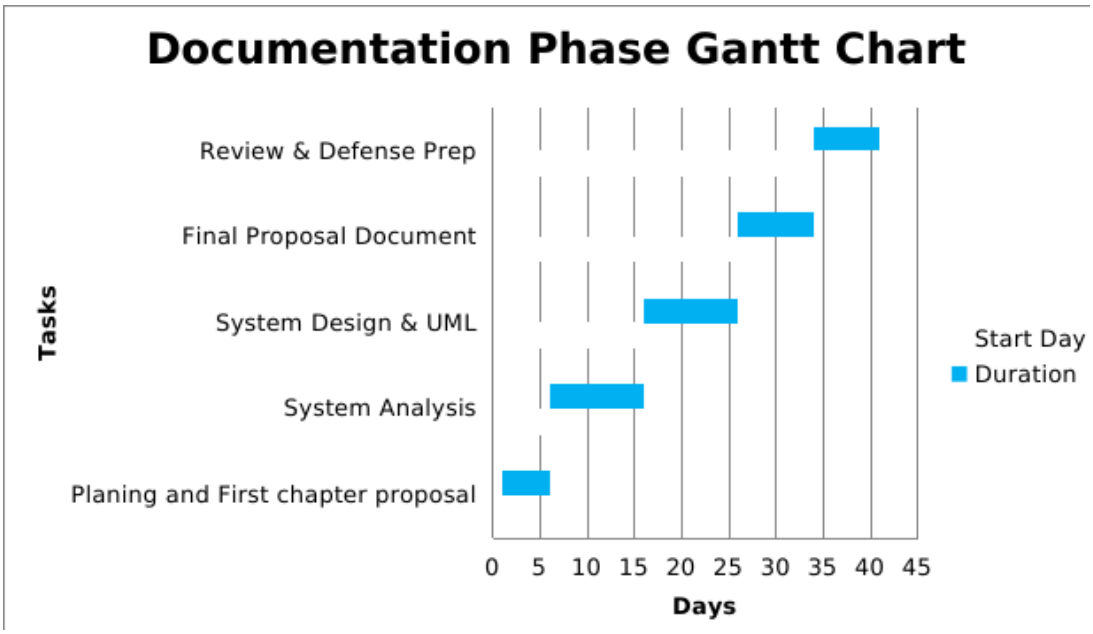


Figure 1.1 Documentation phase Gantt chart

1.9.2 Implementation Phase

The second phase involves the practical realization of the system including backend development, GPS simulation module, real-time communication, frontend UI, testing, and final thesis preparation.

Gantt Chart for Implementation Phase: The following Gantt chart represents the 16-week system implementation timeline:

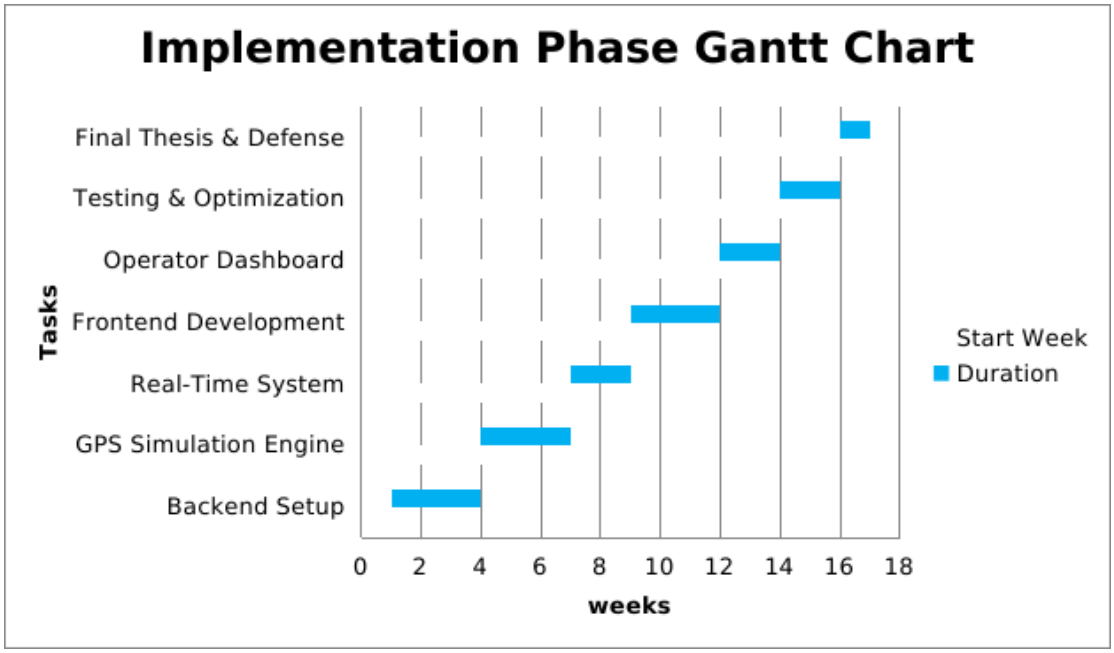


Figure 1.2 Implementation phase Gantt chart

CHAPTER TWO

SYSTEM AND REQUIREMENT ANALYSIS

2.1 Introduction

Requirement analysis is a crucial early phase in system development aimed at understanding and documenting what the system must do to meet user needs. It serves to identify functional expectations, performance criteria, and constraints before design and implementation begin, helping to prevent costly changes later in the project. Requirement analysis ensures clarity and alignment among all participants on what the system should achieve[1].

Why Requirement Analysis is Needed for GizeGo?

For GizeGo, a real-time public transportation tracking and route optimization system for Addis Ababa city, requirement analysis is essential to understand current problems in the transport system and to define what features the system must provide such as real-time vehicle tracking, route optimization, and user interfaces for different users. This step ensures that the developed solution effectively addresses the real needs of passengers, operators, drivers, and administrators, and aligns with the technical and operational context of Addis Ababa's public transport environment. Understanding these requirements helps reduce uncertainty and increases the likelihood that the final system will be useful, usable, and accepted by its users.

Stakeholders:

The primary stakeholders involved in this project include:·

- Passengers: Daily commuters who need reliable, real-time information about public transport arrival times and locations to better plan their journeys.·
- Operators: Public transport operators responsible for scheduling, tracking, and managing buses; they need tools for monitoring fleet performance and optimizing routes.·

Drivers: Individuals who operate the buses; they interact directly with the system's mobile interface for location updates and route guidance.·

System Administrators: The personnel who manage the backend, system configuration, user roles, and overall system health to ensure data integrity and secure access.

Secondary Stakeholders

Transport Authorities (future users): May use analytics and historical data for planning.

Developers: Require clear system requirements for implementation and maintenance.

Each stakeholder's needs were analyzed to ensure the proposed system addresses their expectations effectively. Stakeholders bring diverse needs from real-time visibility for passengers to operational control for admins, making requirement analysis critical to capturing all perspectives.

2.2 Current System

2.2.1 Major Functions of the Current System

The current public transportation system in Addis Ababa operates without any digital tools, real-time tracking, or automated route optimization. Instead, it relies on traditional and manual practices. The major functions of the existing system can be described as follows:

1. Static Scheduling

The public transport system operates using fixed and pre-planned schedules that are manually created by transport operators or local administrators, with buses departing at predetermined times rather than responding to real-time road or traffic conditions. These schedules are infrequently updated and often remain unchanged despite delays, road closures, or variations in passenger demand, limiting operational flexibility.

2. Manual Operations and Coordination

Daily transport operations largely depend on driver decisions and supervisor instructions rather than data generated by an automated system. Route adherence is monitored manually by supervisors stationed at major terminals. Drivers rely on personal judgment to maintain schedules or adjust speed in response to traffic conditions. Communication

between drivers and operators is primarily conducted through phone calls or radio communication instead of a centralized digital monitoring system.

3. Limited or No Real-Time Passenger Information

Passengers have little to no access to live information regarding bus locations or expected arrival times, resulting in significant uncertainty during daily commutes. Commuters often wait for extended periods at bus stops without knowing when the next bus will arrive and must rely on visual spotting, asking other passengers, or estimating arrival times based on experience.

4. Lack of Dynamic Route and Traffic Management

Route management is static and does not adapt to real-time traffic conditions, forcing buses to follow the same paths despite congestion, accidents, or roadblocks. Any adjustments made are informal and based on driver experience or supervisor instructions rather than data-driven decisions. The absence of GPS-based monitoring and route optimization algorithms results in inefficient travel times and poor service reliability.

5. Manual Recording and Reporting

Operational data such as arrival times, delays, fuel usage, and passenger load is recorded manually using paper-based or verbal methods. This approach reduces data accuracy and consistency while increasing the risk of information loss. Consequently, operators are unable to effectively analyze performance trends or make informed operational decisions.

6. Terminal-Based Monitoring

Transport monitoring is largely limited to bus terminals and stations rather than continuous tracking along routes. Supervisors only observe buses during arrival or departure at terminals. Events occurring between terminals remain unknown until the bus reaches the next station, limiting real-time operational awareness.

2.2.2 Problems of the Existing System

The current public transport system in Addis Ababa faces several operational and service-level problems due to the absence of digital tools, real-time tracking, and automated route optimization. These challenges reduce efficiency for operators and create inconvenience for passengers. The major problems are described below:[2].

1. Lack of Real-Time Information

Passengers do not have access to live updates on bus locations, arrival times, or delays, resulting in long and uncertain waiting periods at bus stops. The absence of GPS-based tracking prevents drivers, operators, and passengers from having visibility into real-time bus movement. Consequently, commuting becomes unpredictable, leading to frustration, missed appointments, and low passenger satisfaction.

2. Inefficient Resource Utilization

Fixed schedules and manual decision-making prevent buses from adapting to changing traffic conditions and passenger demand. This results in overcrowding on some routes during peak hours while other routes remain underutilized. The lack of operational data limits operators' ability to redistribute resources effectively, increasing costs and reducing service quality.

3. Poor Decision Support for Operators

Transport operators lack accurate and timely data to support operational decision-making. Without real-time tracking, it is difficult to identify delayed buses, route deviations, or congestion issues.

4. Communication Gaps

Transport system lacks a structured mechanism for communicating real-time updates to passengers and drivers. Information about delays, breakdowns, or schedule changes is rarely shared in a timely manner. This leads to confusion, poor coordination, and reduced trust among all stakeholders.

5. Limited Monitoring and Accountability

Operational monitoring is largely restricted to terminals, leaving activities between stations unobserved. Manual reporting methods are prone to errors and data loss, making it difficult to track performance or enforce service standards. This lack of transparency reduces accountability and limits opportunities for service improvement.

2.3 Requirement Gathering

Requirement gathering is a critical phase in system development that involves identifying, collecting, and analyzing user needs and system expectations. For the GizeGo project, requirement gathering was conducted to clearly understand the challenges faced by public

transportation users in Addis Ababa and to define the functionalities required for a simulation-based real-time bus tracking and route optimization system.

2.3.1 Requirement Gathering Methodologies

Multiple requirement gathering methodologies were used to ensure completeness and accuracy:

a) Literature Review

Relevant research papers, journals, and case studies related to real-time transportation systems, route optimization, GPS simulation, and intelligent transportation systems were reviewed. This helped identify standard features, best practices, and technological approaches suitable for the project.

b) Observation

The team observed daily public transportation operations at major bus stops in Addis Ababa. This included monitoring passenger waiting times, lack of schedule visibility, congestion effects, and information gaps.

c) Informal Interviews

Informal discussions were held with passengers and public transport users to understand common commuting challenges, expectations, and desired system features such as real-time tracking and ETA visibility.

d) Requirement Analysis of Existing Systems

Existing transportation applications such as Moovit and Google Transit were analyzed to understand their core functionalities and limitations within the local context.

2.3.2 Results Found

The requirement gathering process revealed several key findings:

- Passengers experience high uncertainty due to the absence of real-time information.
- Operators lack centralized tools for fleet monitoring and route optimization.
- Static routes and schedules contribute to inefficiency during congestion.

- There is strong demand for a mobile-based, easy-to-use tracking system.
- A simulation-based solution is suitable for demonstrating system feasibility without real GPS hardware.

These findings directly influenced the design and scope of the proposed GizeGo system.

2.4 Proposed System

2.4.1 Overview

The proposed system, GizeGo, is a simulation-based real-time public transportation tracking and route optimization system designed for Addis Ababa city. The system simulates the movement of buses using generated GPS data and provides real-time tracking through a mobile application and an operator dashboard.

GizeGo integrates OpenStreetMap (OSM) for route visualization, uses WebSocket communication for live updates, and applies the A* pathfinding algorithm to optimize routes based on simulated traffic conditions. The system supports bilingual interfaces (Amharic and English) and is designed to be scalable for future real-world deployment.

2.4.2 Functional Requirements

FR1. User Account & Authentication.

- The system shall allow users to register an account using a phone number or email.
- The system shall allow users to securely log in and log out.
- The system shall support role-based access control for passengers, drivers, ticket vendors, and administrators.
- The system shall allow users to update their profile information.

FR2. Route Management

- The system shall allow creation, editing, and deletion of bus routes.
- The system shall load route map data from OpenStreetMap (OSM).
- The system shall store route polylines and related metadata in the database.
- The system shall display available routes and their corresponding stops.

FR3. Bus & Fleet Management (Simulation-Based)

- The system shall simulate the movement of 10-20 virtual buses/minibuses.
- The system shall generate continuous GPS updates for each bus every 2–5 seconds.
- The system shall simulate variable traffic conditions that affect bus speed and delays.
- The system shall update each bus's real-time status, including speed, direction, and location.

FR4. Real-Time Tracking.

- The system shall display the live location of all buses on a map interface.
- The system shall update bus positions in real time using WebSocket communication (e.g., Socket.io).
- The system shall show the status of each bus (moving, delayed, stopped).
- The system shall allow users to view the detailed path of any selected bus.

FR5. Route Optimization

- The system shall compute optimal routes using the A* shortest path algorithm.
- The system shall consider traffic weights when generating optimized routes.
- The system shall recalculate the route if traffic conditions change.
- The system shall present alternative route suggestions when available.

FR6. ETA Computation

- The system shall calculate bus ETAs based on distance, speed, and traffic conditions.
- The system shall update ETAs dynamically as new GPS data arrives.
- The system shall display ETAs for upcoming stops or user-selected points.

FR7. Notifications & Alerts.

- The system shall send real-time notifications for delays, congestion, or route changes.
- The system shall notify users when ETA values change significantly.
- The system shall provide system-wide alerts for operational issues when necessary.

FR8. Dashboard & Analytics

- The system shall provide a dashboard displaying real-time fleet status.
- The system shall show analytics such as average travel time, delays, and congestion patterns.
- The system shall provide visualizations, including charts and graphs.
- The system shall support filtering analytics by route, bus, or time period.
- The system shall generate system performance and operational reports.

FR9. Data Storage & Management

- The system shall store GPS history, route data, bus logs.
- The system shall allow exporting data in standard formats (CSV, JSON).
- The system shall archive historical data for analytics and research.
- The system shall maintain data integrity and prevent unauthorized access.

FR10. Language & Localization

- The system shall support English and Amharic language interfaces.
- The system shall allow switching language at any time without re-login.

FR11. Error Handling

- The system shall detect missing or invalid GPS updates and handle them gracefully.
- The system shall notify users when network or server errors occur.
- The system shall log all errors for debugging and system improvement.

2.4.3 Non-Functional Requirements

a) Performance

- The system shall update bus locations every 2-5 seconds.
- Route optimization calculations shall complete within acceptable response time.

b) Usability

- The user interface shall be intuitive and easy to use.
- The mobile application shall support bilingual operation.
- The system shall be accessible to users with minimal technical knowledge.

c) Reliability

- The system shall maintain stable real-time communication.
- Temporary network failures shall not crash the system.

d) Security

- User authentication data shall be securely stored.
- Unauthorized access to system functions shall be prevented.

e) Scalability

- The system shall allow additional routes and buses to be added easily.
- The architecture shall support future integration with real GPS devices.

f) Maintainability

- The system shall use modular design for easy maintenance and updates.
- Error logs shall be maintained for debugging purposes.

2.5 Assumptions and Project Limitations

This section describes the assumptions made during the development of the proposed system and the constraints that limit the scope and implementation of the project. Identifying these factors helps clarify the conditions under which the system is designed and evaluated.

2.5.1 Assumptions

The following assumptions are made for the successful development and operation of the GizeGo system:

1. It is assumed that users have access to smartphones with basic internet connectivity.
2. It is assumed that OpenStreetMap (OSM) provides sufficiently accurate route and road data for Addis Ababa.
3. It is assumed that simulated GPS data can reasonably represent real-world bus movement behavior.
4. It is assumed that users are familiar with basic mobile application usage.
5. It is assumed that traffic conditions can be approximated using simulated traffic weights and time-based variations.
6. It is assumed that the system will be used mainly for demonstration and academic evaluation purposes.

2.5.2 Project Limitations

The development of the GizeGo simulation-based real-time bus tracking and route optimization system is influenced by several constraints that shape the scope, design decisions, and implementation approach. These constraints must be acknowledged to ensure that the system remains feasible within academic, technical, and operational boundaries.

1. Technical Constraints

The system is developed as a simulation-based solution without physical GPS hardware, relying entirely on internally generated virtual coordinates and movement patterns. OpenStreetMap is used as the sole mapping source due to cost limitations, which may affect route accuracy in some areas. In addition, the application must be optimized to run on low-end Android devices and handle real-time communication delays caused by unstable network conditions.

2. Time Constraints

Project development and delivery are strictly governed by the academic calendar, requiring all design, implementation, testing, and documentation activities to be completed within fixed deadlines. This limits the depth of feature expansion and iterative refinement.

3. Financial Constraints

The project is implemented with a zero-budget approach, prohibiting the use of GPS hardware, paid APIs, cloud hosting, or commercial monitoring tools. All development relies on free-tier services, open-source technologies, and locally developed simulation components.

4. Scope Restriction

To maintain feasibility within time and resource limitations, the system simulates only 10–20 virtual buses operating on 5–10 predefined routes. Large-scale deployment and real-world integration are intentionally excluded from the current scope.

5. Environmental Constraints

Real-time tracking depends on internet connectivity, which may be slow or unstable, requiring the system to handle delays, disconnections, and packet loss gracefully. Additionally, limited hardware resources and the absence of professional testing environments constrain performance testing and system validation.

2.6 System model

2.6.1 Scenario

2.6.1.1 As is scenario

Addis Ababa's public transportation currently operates without any centralized real-time tracking system. Passengers rely on guesswork to know when buses will arrive, often waiting long periods with no information on delays or congestion. Drivers follow static routes without dynamic updates or optimized path suggestions. Transport operators have no dashboard to monitor fleet movement, bus performance, or traffic conditions.

There is no simulation system, so transport authorities cannot test alternative routes or evaluate network performance. Communication between passengers, drivers, and operators is minimal. When delays occur, passengers are unaware until they reach the bus station. There is also no historical database storing route performance, travel times, or operational issues.

Overall, the existing system is characterized by inefficiency, unpredictability, lack of digital tools, and an absence of data-driven decision-making.

2.6.1.2 Visionary Scenario

A passenger opens the GizeGo mobile application and immediately sees real-time bus locations moving smoothly on the map. The app shows ETAs that update dynamically as traffic conditions change. Passengers receive notifications when the bus they are waiting for is delayed or rerouted.

An operator logs in to the system dashboard and views the status of all buses on the network. Color-coded indicators show which buses are delayed, stopped, or operating normally. The operator can monitor congestion levels, average travel time, and system-wide performance.

Behind the scenes, the system simulates 10–20 buses that update their GPS positions every few seconds. The A* algorithm continuously computes the most efficient routes based on simulated traffic weights. If congestion increases on a road segment, the system automatically recalculates an optimized route and updates the bus path.

Administrators can manage routes, buses, and users through a secure backend system. The entire platform runs bilingually (Amharic and English), enabling easy adoption by the public.

This visionary future demonstrates a smart, data-driven, real-time transport ecosystem that reduces uncertainty, improves reliability, and enhances city mobility.

2.6.1.3 Evaluation Scenario

A group of test users including passengers, operators, and administrators log in to the system to evaluate its functionality.

Passenger Evaluation Scenario

1. A test user registers using a phone number or email and logs in.
2. They select a bus route and begin tracking a simulated bus.
3. They observe live GPS updates every 2–5 seconds on the map.
4. They compare the displayed ETA with actual simulated arrival times to evaluate accuracy.
5. They receive notifications when the bus slows down due to simulated traffic.

Operator Evaluation Scenario

1. An operator logs in to the dashboard.
2. They monitor all buses and check whether the system correctly shows movement, delays, and stops.
3. They use the analytics panel to view travel times and congestion patterns.
4. They test route optimization by introducing simulated traffic and confirming that the A* algorithm suggests alternative routes.
5. They export route performance data and verify if CSV/JSON reports are correctly produced.

Admin Evaluation Scenario

1. The admin creates, edits, and deletes routes.
2. They test user management, localization, and API response validation.
3. They intentionally send invalid GPS data to confirm that the system handles errors gracefully.

2.6.1.4 Training Scenario

- Passenger Training Scenario

The user installs the GizeGo mobile app and is guided through a short in-app tutorial. The tutorial shows how to:

· create an account

· choose a route

· view live bus locations

· check ETAs

· switch between Amharic and English

- Operator Training Scenario

Operators attend a training session where they are introduced to:

· logging into the dashboard

· interpreting color-coded bus statuses (moving, delayed, stopped)

· monitoring congestion levels

· using analytics tools

· generating reports

· managing fleet simulation settings

They complete a guided exercise where they observe live bus data and test route optimization under simulated traffic.

- Administrator Training Scenario

Admins receive training on:

· route creation, modification, and deletion

· user access management

· API usage and validation rules

· handling errors and reviewing system logs

· managing data exports and archives

They work through practice scenarios such as creating a new route, testing WebSocket communication, and validating ticket transactions.

The training scenario ensures that all user roles become confident and competent in operating the entire GizeGo system.

2.6.2 Use case model

2.6.2.1 Use case diagram

Use Case Diagrams are used to depict the functionality of a system or a part of a system. They are widely used to illustrate the functional requirements of the system and its interaction with external agents(actors) [3].

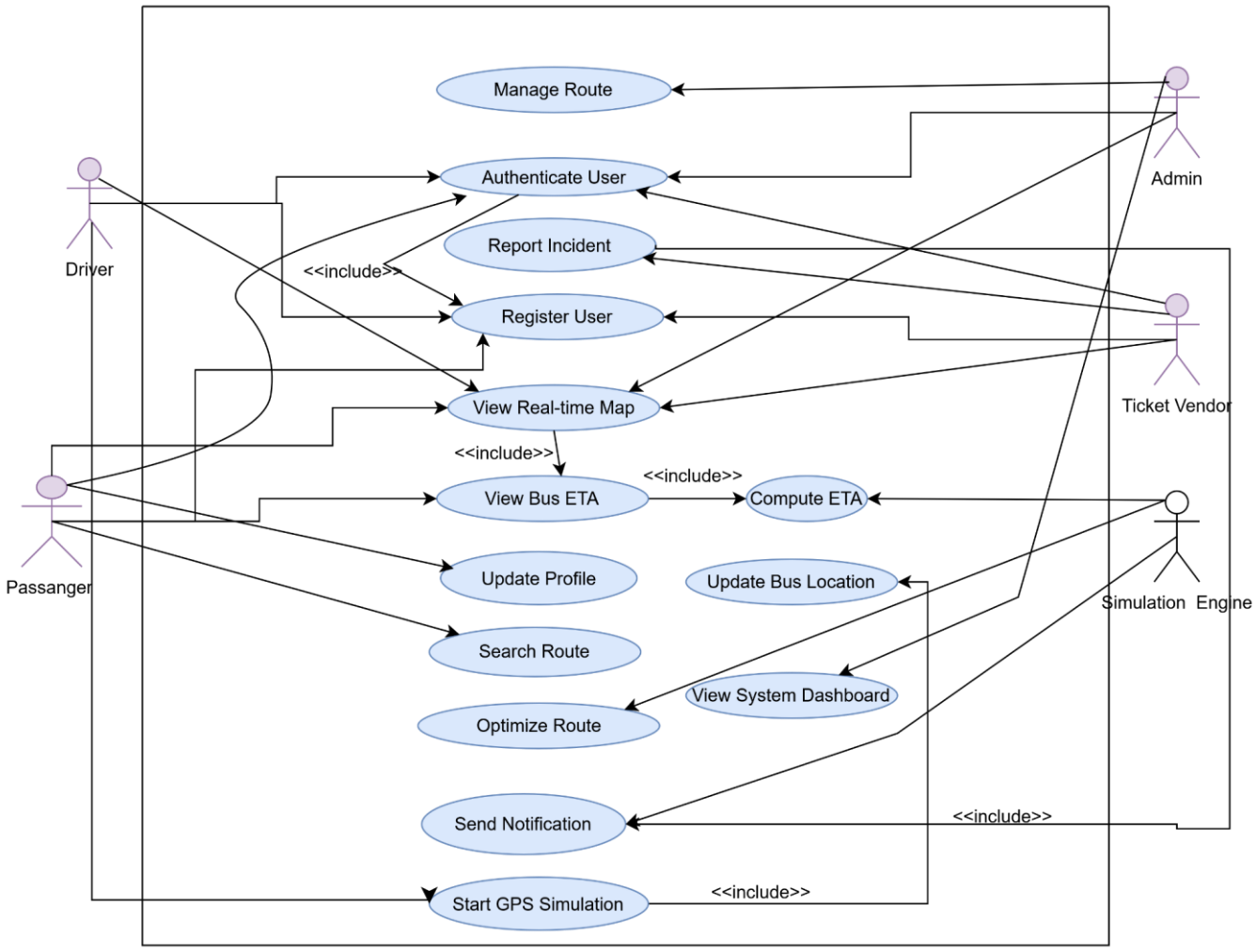


Figure 2.1 Use case diagram

Actors Description

1. Passenger

A passenger is a public user who uses the system to obtain real-time information about buses.

2 .Administrator

The administrator is responsible for managing and controlling the entire system.

3. Ticket Vendor

The ticket vendor represents on-site staff at bus stations or on buses who report incidents and operational issues to the system.

4. Driver

The driver represents the person operating the bus and interacting with the system for operational updates.

5. Simulation Engine

A logical external actor that generates simulated bus movement and location data in place of real GPS devices. It triggers periodic location updates and automated system events such as delay detection and notifications.

2.6.2.2 Description of use case model

Table 2.1 Description for Register User use case

Use Case ID	UC01
Use Case Name	Register User
Actor	Passenger, Driver, Ticket Vendor
Goal	Allow a new user to create an account
Pre-conditions	· The user is not already registered. · The system is available.
Post conditions	A new user account is created and stored in the database
Main Flow	1. The user opens the registration page. 2. The user enters required information (name, phone/email, password). 3. The system validates the input. 4. The system creates a new account and stores it in the database. 5. The system confirms successful registration.
Alternative Flow	A1. Invalid Input: 3a. If the data is invalid, the system returns an error.

	3b. The user corrects input and retries.
Quality Requirements	· Response time for registration should be less than 5 seconds. · Input validation must prevent duplicate accounts.

Table 2.2 Description for Authenticate User use case Table

Use Case ID	UC02
Use Case Name	Authenticate User
Actor	Passenger, Driver, Ticket Vendor, Administrator
Goal	Allow a user to log in to the system
Pre-conditions	· The user already has an account. · Credentials are valid.
Post conditions	User is authenticated and granted access based on role
Main Flow	1. The user provides email/phone and password. 2. System validates credentials. 3. System grants access and loads appropriate dashboard
Alternative Flow	A1. Incorrect Credentials: System displays an error and denies access

Quality Requirements	· Authentication must complete within 4 seconds. · System must prevent brute-force login attempts

Table 2.3 Description for Update Profile use case

Use Case ID	UC03
Use Case Name	Update Profile
Actor	Passenger
Goal	Modify user profile information
Pre-conditions	The user is logged in.
Post conditions	Updated profile information is saved successfully
Main Flow	1. The user navigates to the profile page. 2. User updates fields (name, phone, language preference). 3. System validates changes. 4. The system saves updates.
Quality Requirements	· All changes should be reflected instantly.

Table 2.4 Description for View Real-Time Map use case

Use Case ID	UC04
Use Case Name	View Real-Time Map
Actor	Passenger, Administrator, Driver, Ticket Vendor
Goal	View all active buses on a map
Pre-conditions	· The system has active GPS feeds. · The user is authenticated.
Post conditions	The user views an updated map with bus locations.
Main Flow	1. The user opens the real-time map page. 2. The system retrieves bus locations. 3. The system displays buses, routes, and status. 4. The system updates bus positions every 2–5 seconds.
Quality Requirements	· Map updates must occur with < 1-second latency. · Bus markers must remain accurate and synchronized

Table 2.5 Description for Search Route use case

Use Case ID	UC05
Use Case Name	Search Route
Actor	Passenger
Goal	Allow passenger to search for a specific route
Pre-conditions	The system contains available routes.
Post conditions	User receives matching routes and details
Main Flow	1. User inputs source and destination. 2. The system retrieves matching routes. 3. System displays route details and active buses
Quality Requirements	Search results must load within 4 seconds

Table 2.6 Description for View Bus ETA use case

Use Case ID	UC06
Use Case Name	View Bus ETA
Actor	Passenger
Goal	View estimated arrival time of a selected bus
Pre-conditions	· Bus location is available. · The ETA computation engine is running.
Post conditions	ETA is displayed to the user
Main Flow	1. The passenger selects a bus on the map. 2. System retrieves bus location and route. 3. The system computes ETA. 4. ETA is displayed.
Quality Requirements	ETA accuracy should be within ±10 seconds of simulated data

Table 2.7 Description for Report Incident use case

Use Case ID	UC07
Use Case Name	Report Incident
Actor	Ticket Vendor
Goal	Notify the system when an accident, delay, or problem occurs
Pre-conditions	The vendor is authenticated.
Post conditions	Incident is logged and relevant users are notified
Main Flow	1. Ticket Vendor selects "Report Incident". 2. Vendor selects type (delay, accident, broken bus). 3. The vendor adds details or notes. 4. The system stores incidents in the database. 5. System broadcasts notification to passengers & admin
Alternative Flow	A1. Network Failure: System retries sending incident after connectivity is restored
Quality Requirements	· Incident should be registered in < 3 seconds. · Notifications should reach users with < 2-second delay

Table 2.8 Description for Start GPS Simulation use case

Use Case ID	UC08
Use Case Name	Start GPS Simulation
Actor	Driver
Goal	Begin simulated movement of a bus on its route
Pre-conditions	· The driver is assigned a route. · Bus is marked as active
Post conditions	Bus sends periodic GPS updates to backend
Main Flow	1. The driver opens the dashboard. 2. The driver selects “Start Simulation.” 3. The system begins generating GPS points. 4. System updates backend every 2–5 seconds
Quality Requirements	GPS updates must not exceed 5-second intervals

Table 2.9 Description for Manage Routes use case

Use Case ID	UC09
Use Case Name	Manage Routes
Actor	Administrator
Goal	Add, update, or delete transport routes
Pre-conditions	Admin is logged in.
Post conditions	The route list is updated and saved.
Main Flow	1. Admin navigates to route management. 2. Admin selects create/edit/delete. 3. The system validates data. 4. System updates route database.
Quality Requirements	Route updates should be processed within 3 seconds.

Table 2.10 Description for View System Dashboard use case

Use Case ID	UC10
Use Case Name	View System Dashboard
Actor	Administrator
Goal	Monitor real-time status of fleet and operations.
Pre-conditions	Real-time data streams are active.
Post conditions	Admin dashboard shows current analytics.
Main Flow	1. Admin opens the dashboard. 2. The system retrieves bus, route, and incident data. 3. The system renders charts and analytics.
Quality Requirements	Dashboard metrics should refresh every 5 seconds.

Table 2.11 Description for Update Bus Location use case

Use Case ID	UC11
Use Case Name	Update Bus Location
Actor	Simulation engine
Goal	Update bus location using simulated GPS points
Pre-conditions	GPS simulation is running.
Post conditions	The latest bus location is stored.
Quality Requirement	Must support high-frequency updates (2–5 seconds)

Table 2.12 Description for Optimize Route use case

Use Case ID	UC12
Use Case Name	Optimize Route
Actor	Simulation Engine
Goal	Compute best route considering distance + traffic.
Pre-conditions	Route and traffic data are available.
Post conditions	An optimized route is generated.
Main Flow	1. The system retrieves route data. 2. System applies A* algorithm. 3. The system stores optimized routes.
Quality Requirement	The algorithm should compute results < 3 seconds.

Table 2.13 Description for Compute ETA use case

Use Case ID	UC13
Use Case Name	Compute ETA
Actor	Simulation Engine
Goal	Estimate arrival time of bus.
Pre-conditions	Bus location and speed are known.
Post conditions	ETA values are updated.
Quality Requirement	ETA error should be minimized relative to simulation.

Table 2.14 Description for Send Notifications use case

Use Case ID	UC14
Use Case Name	Send Notifications
Actor	Simulation Engine
Goal	Notify users of incidents or delays.
Pre-conditions	Incident or delay exists.
Post conditions	Notifications are delivered.
Main Flow	1. The system identifies affected users. 2. The system sends notifications. 3. System logs delivery status.

2.6.3 Dynamic model

2.6.3.1 Sequence diagram

A sequence diagram simply depicts interaction between objects in a sequential order, i.e. the order in which these interactions take place. It also describes how and in what order the objects in a system function. [3].

The following are Sequence diagram for each use cases

A. Register User

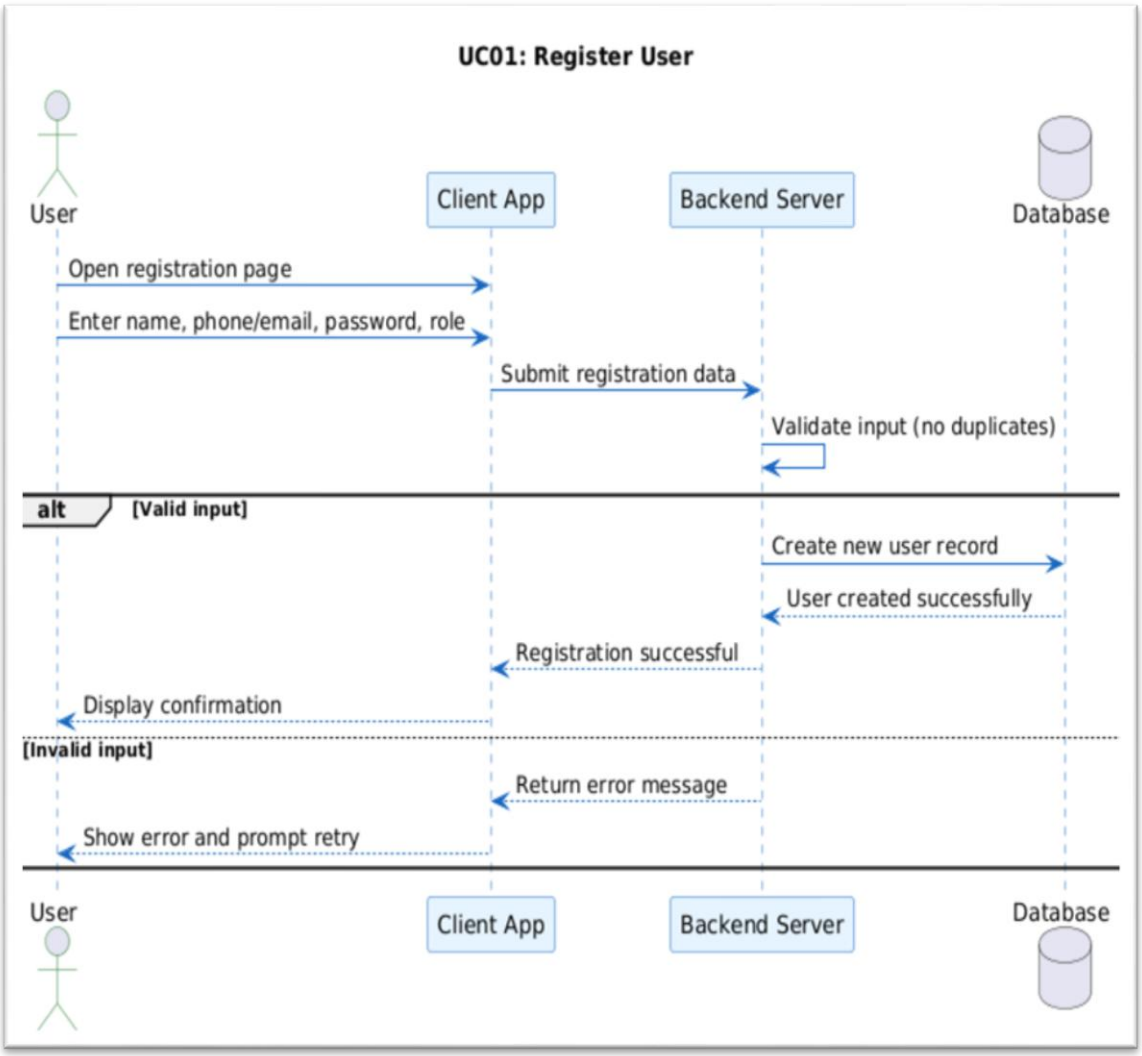


***Figure 2.2 Sequence diagram for* Register User**

B. Authenticate User

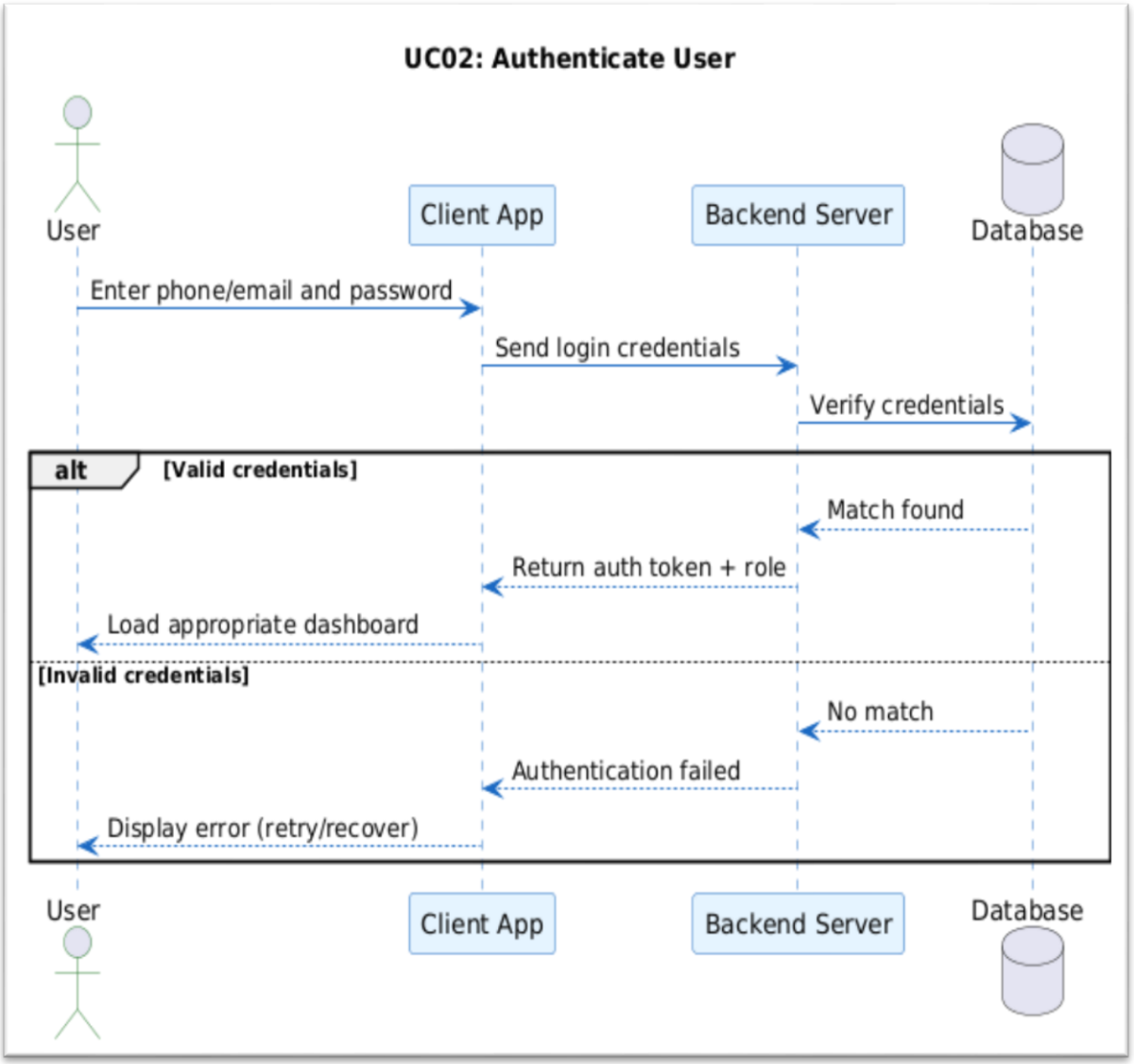


Figure 2.3 Sequence diagram for Authenticate User

C. Update Profile

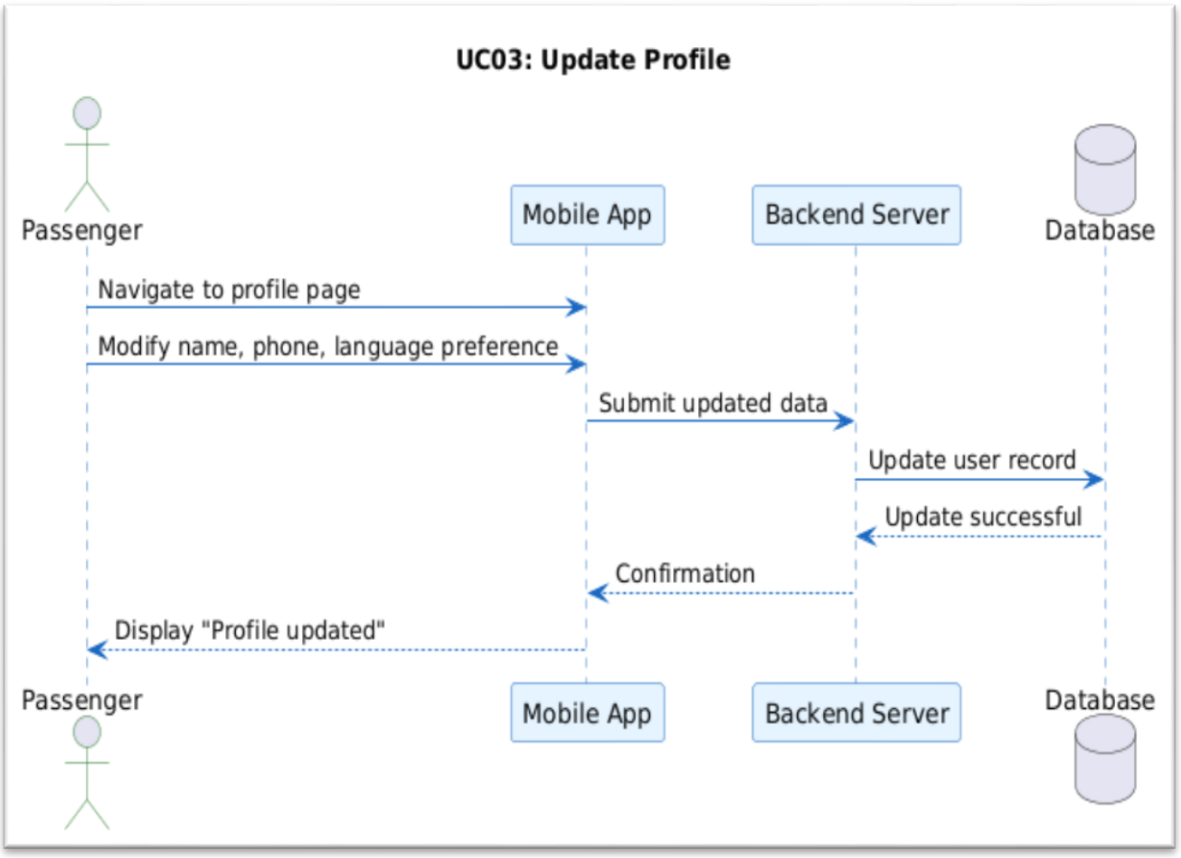


Figure 2.4 Sequence diagram Update Profile

D. View Realtime map

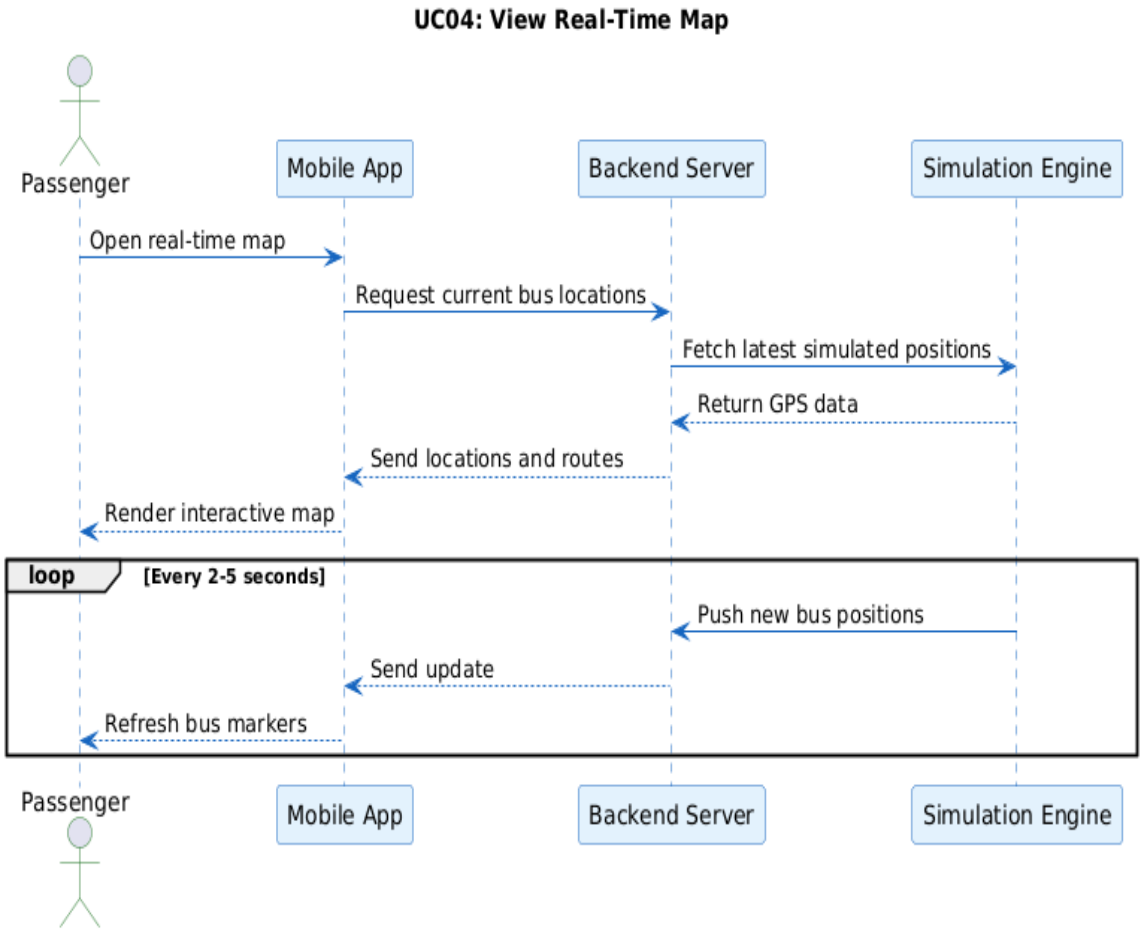


Figure 2.5 Sequence diagram for View Real-time map

F. Search Route

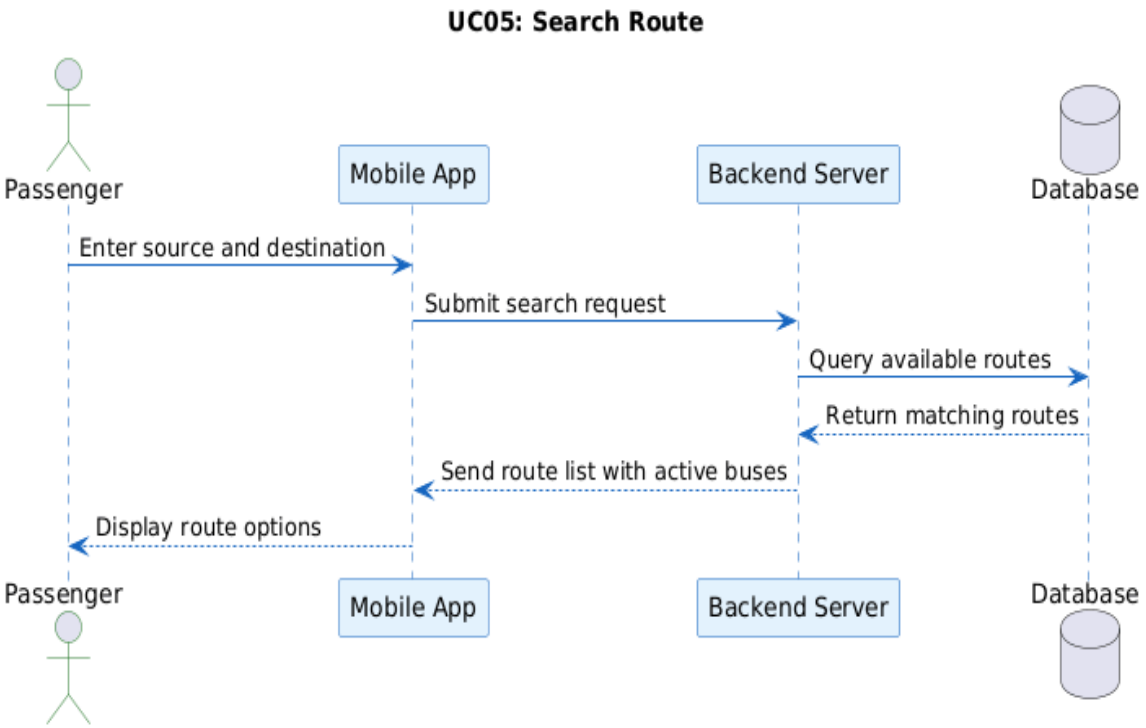


Figure 2.6 Sequence diagram for Search Route

F. View Bus ETA

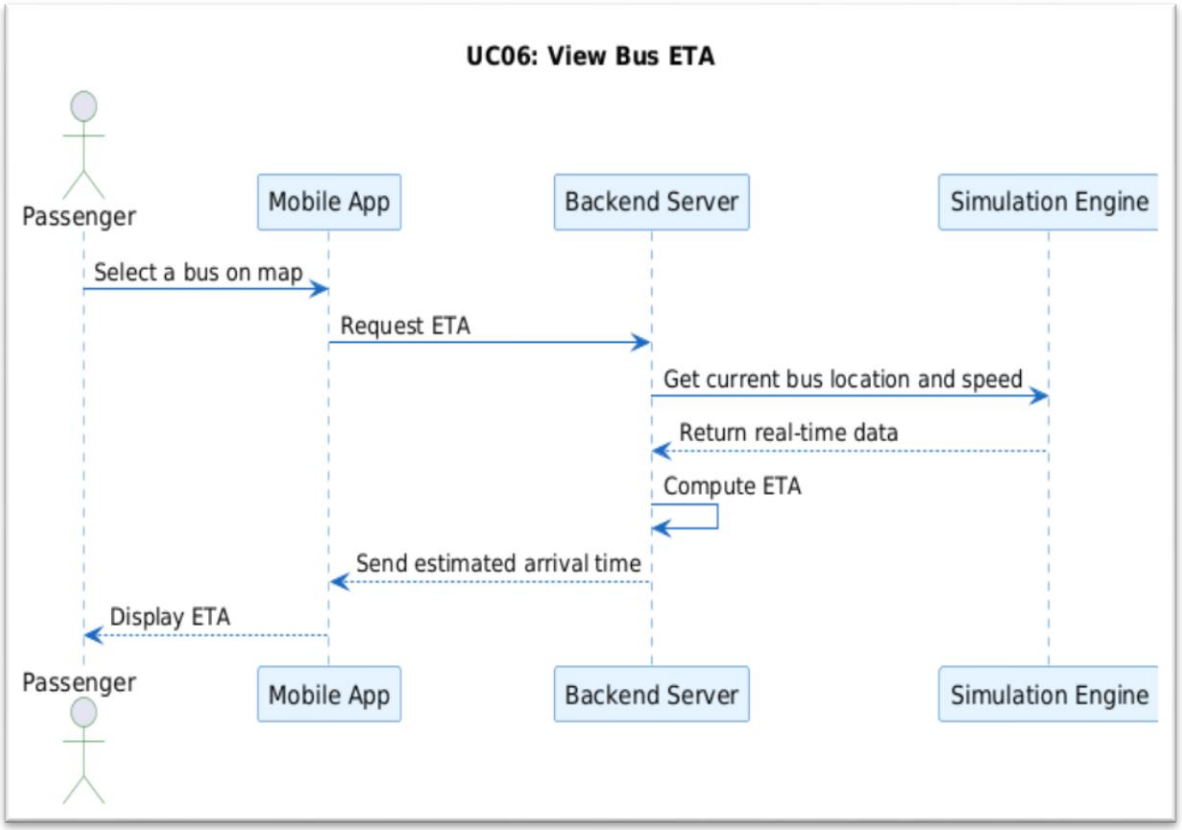


Figure 2.7 Sequence diagram for View Bus ETA

G. Report Incident

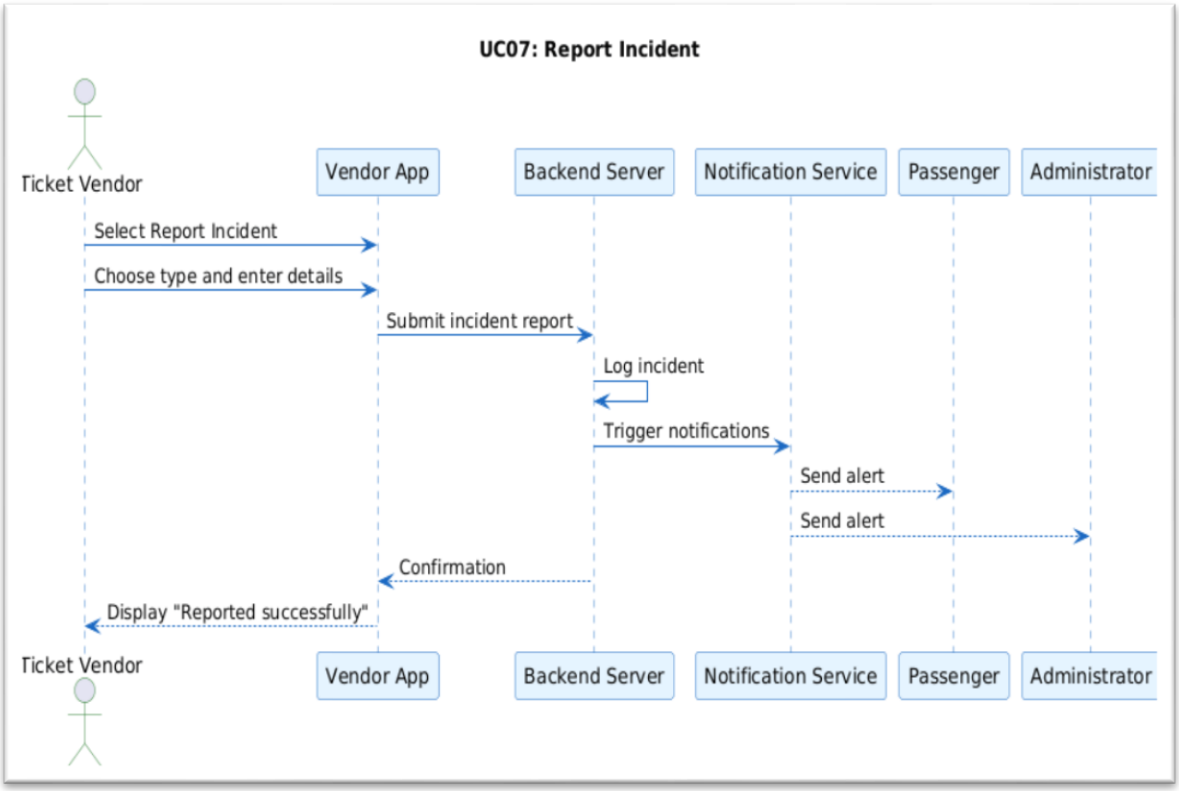


Figure 2.8 Sequence diagram for Report Incident

H. Start GPS Simulation

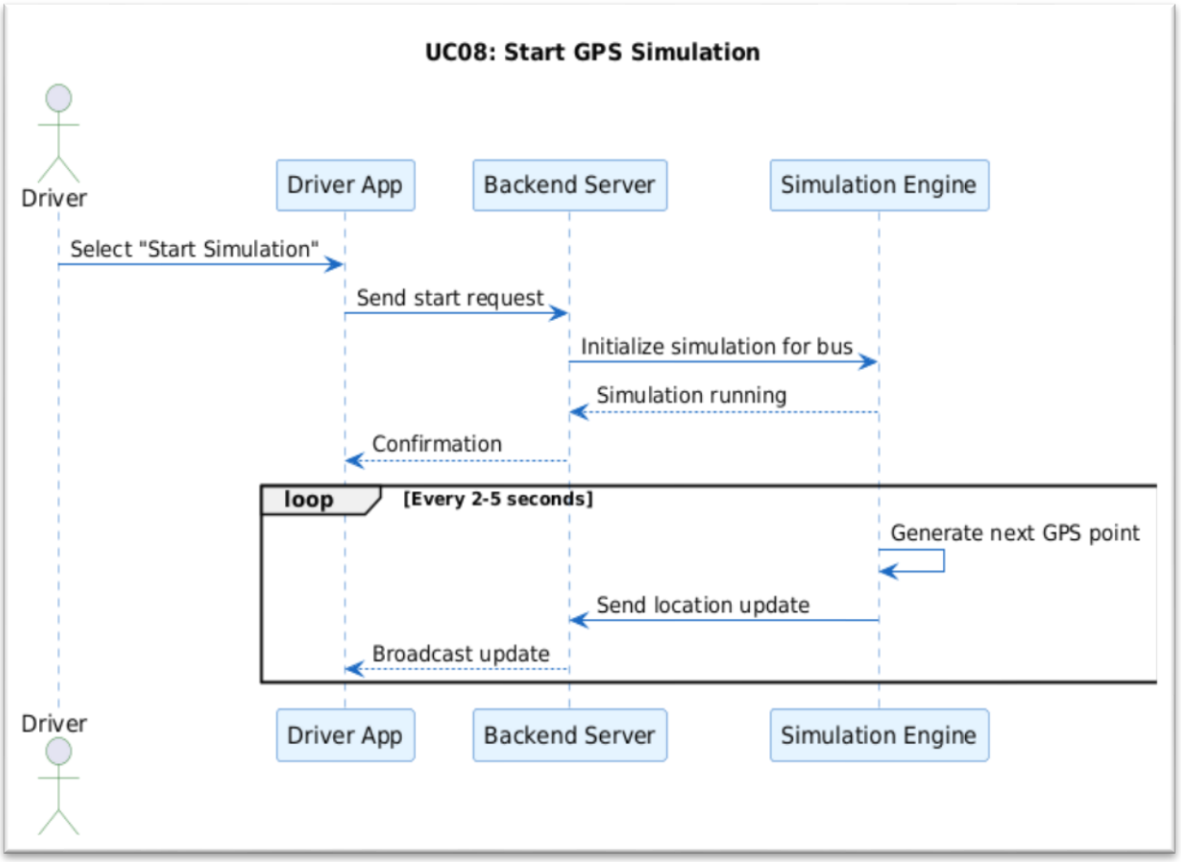


Figure 2.9 Sequence diagram for start GPS Simulation

I. Manage Route

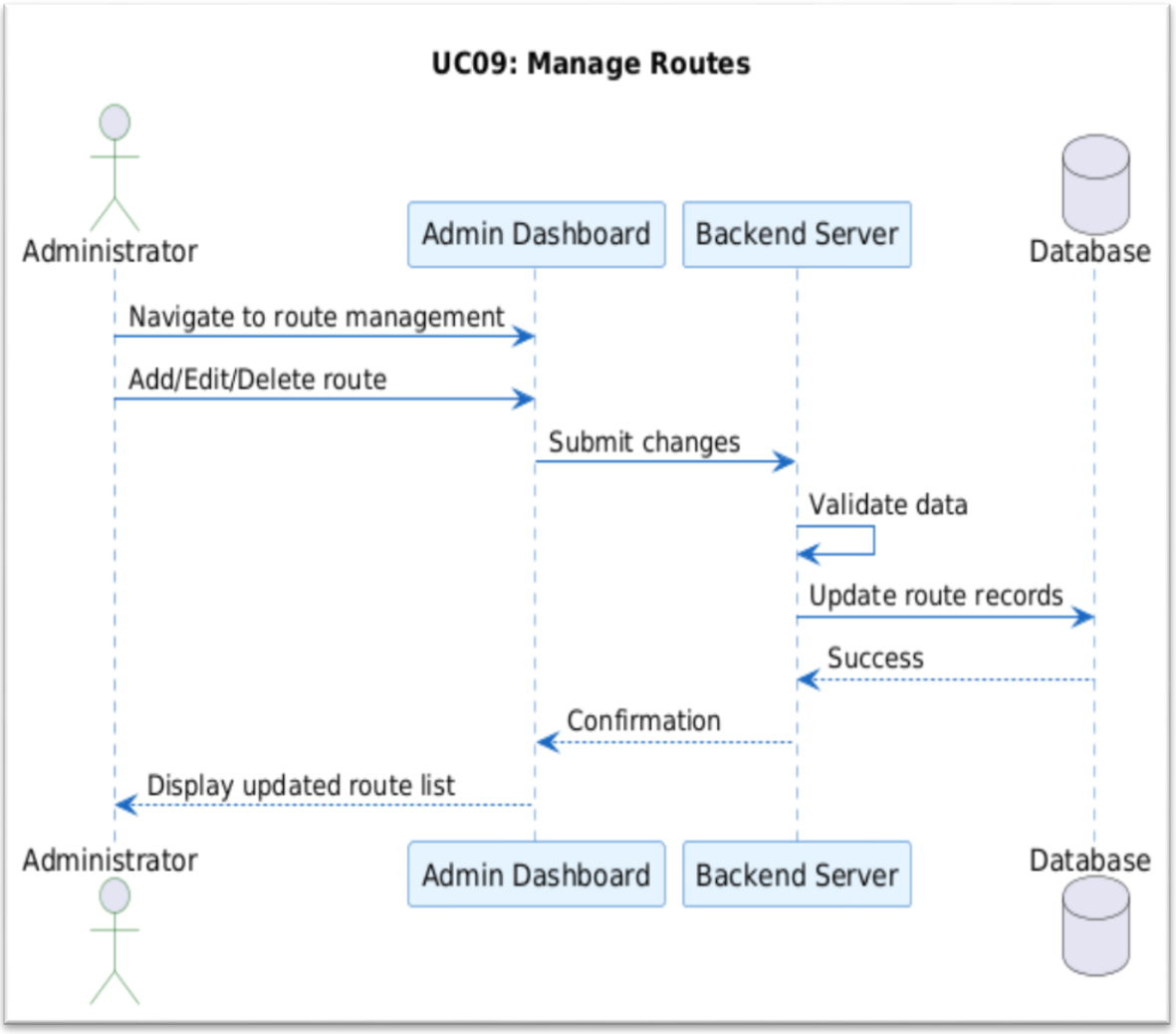


Figure 2.10 Sequence diagram for Mange Route

J. View System Dashboard

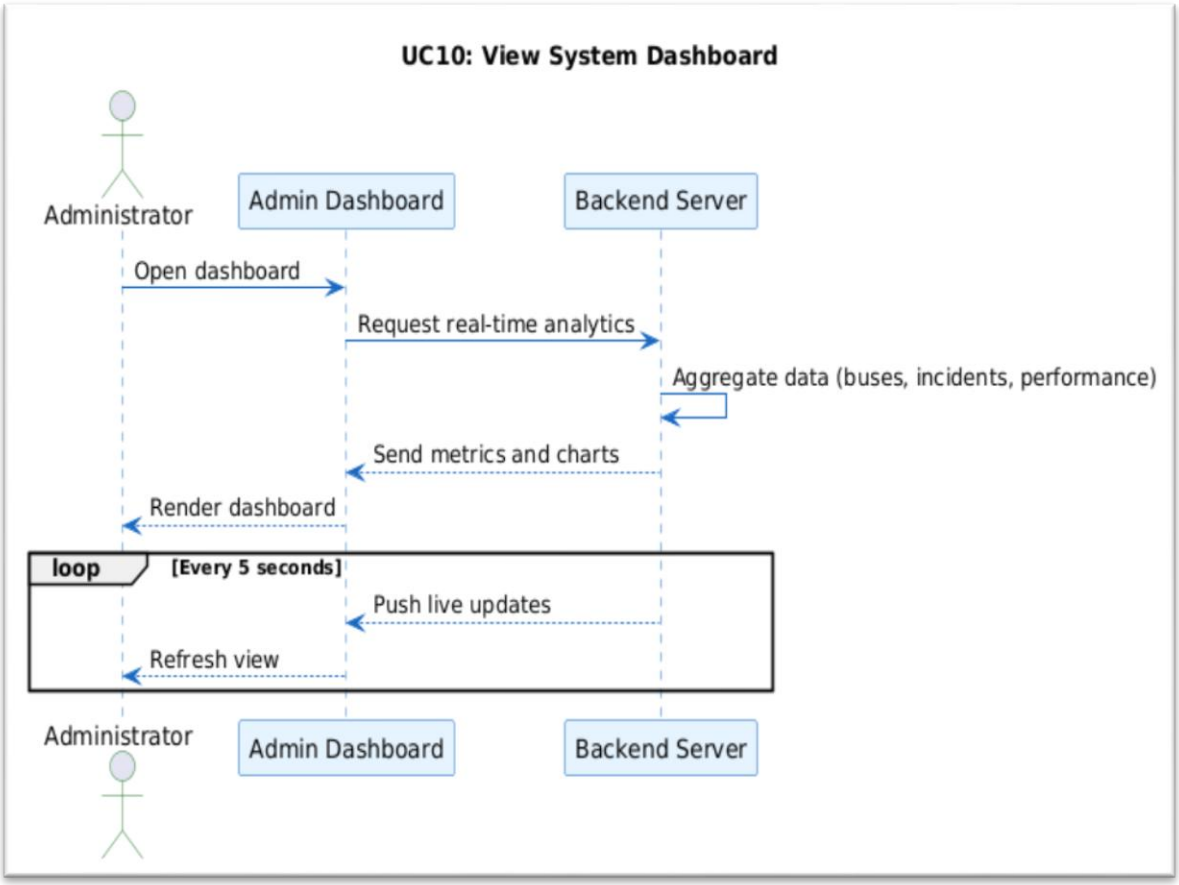


Figure 2.11 Sequence diagram for View System Dashboard

K. Update Bus Location

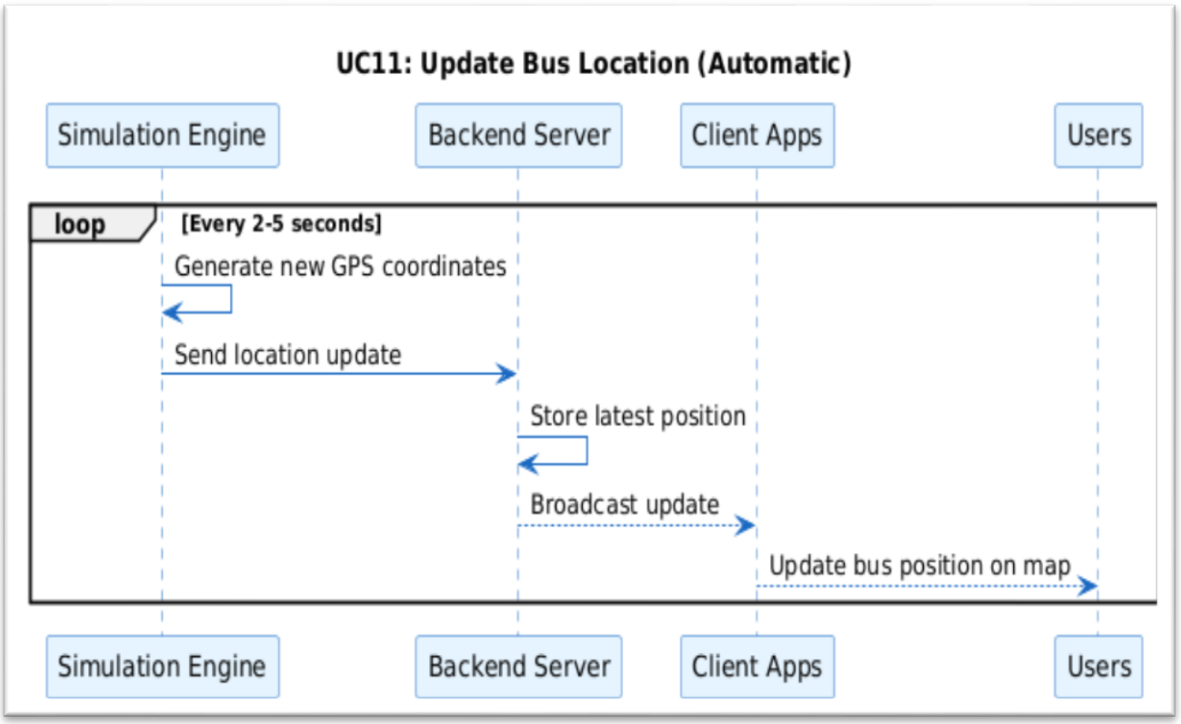


Figure 2.12 Sequence diagram for Update Bus Location

L. Optimize Route

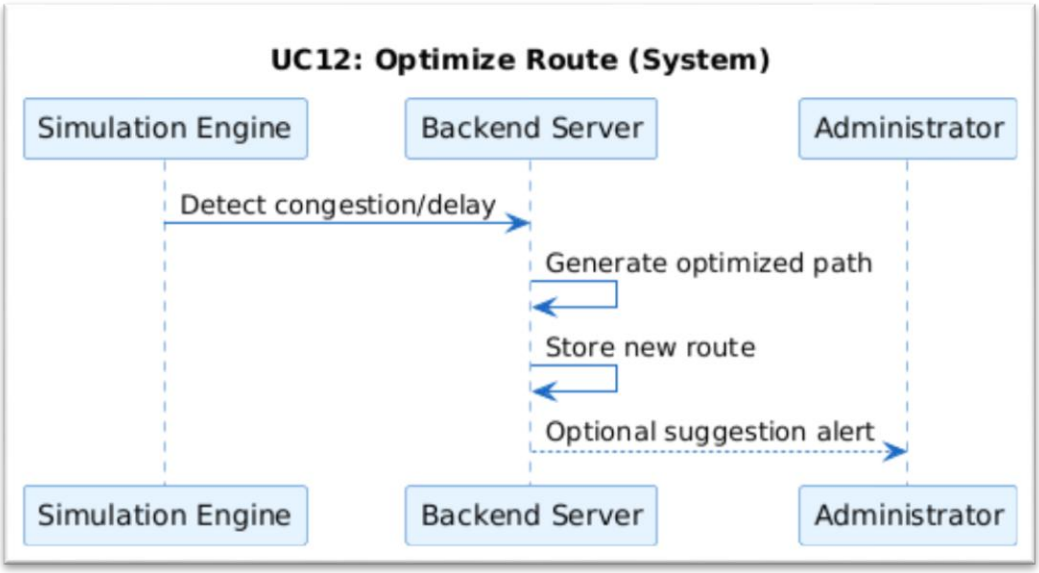


Figure 2.13 Sequence diagram for Optimize Route

M. Compute ETA

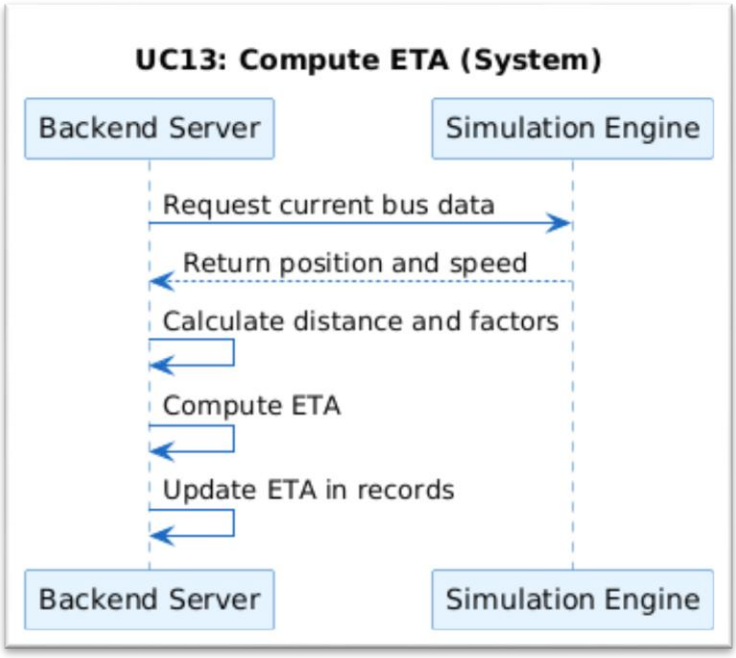


Figure 2.14 Sequence diagram for Compute ETA

N. Send Notification

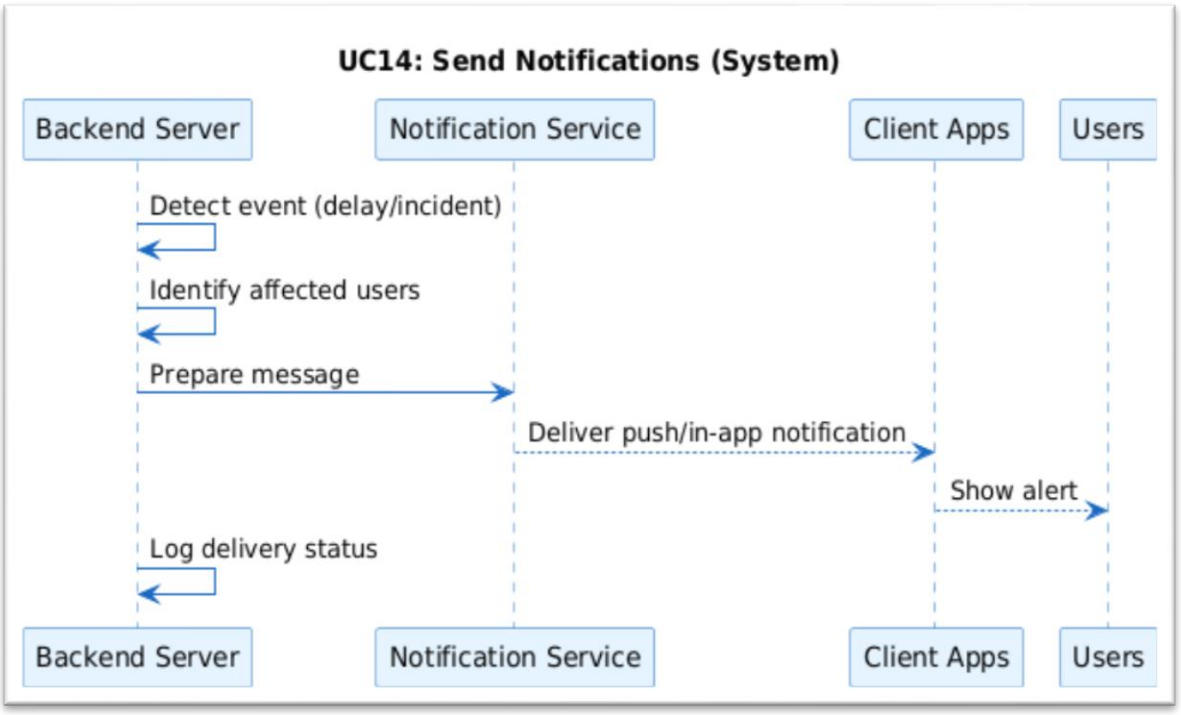


Figure 2.15 Sequence diagram for Send Notification

2.6.3.2 Activity diagram

We use Activity Diagrams to illustrate the flow of control in a system. We can also use an activity diagram to refer to the steps involved in the execution of a use case. It focuses on the condition of flow and the sequence in which it happens [3].

Based on this, the activity diagram presented below describes the step-by-step execution of the selected use case, highlighting the interactions between actors and the system, as well as the conditions that control the flow of activities from start to completion.

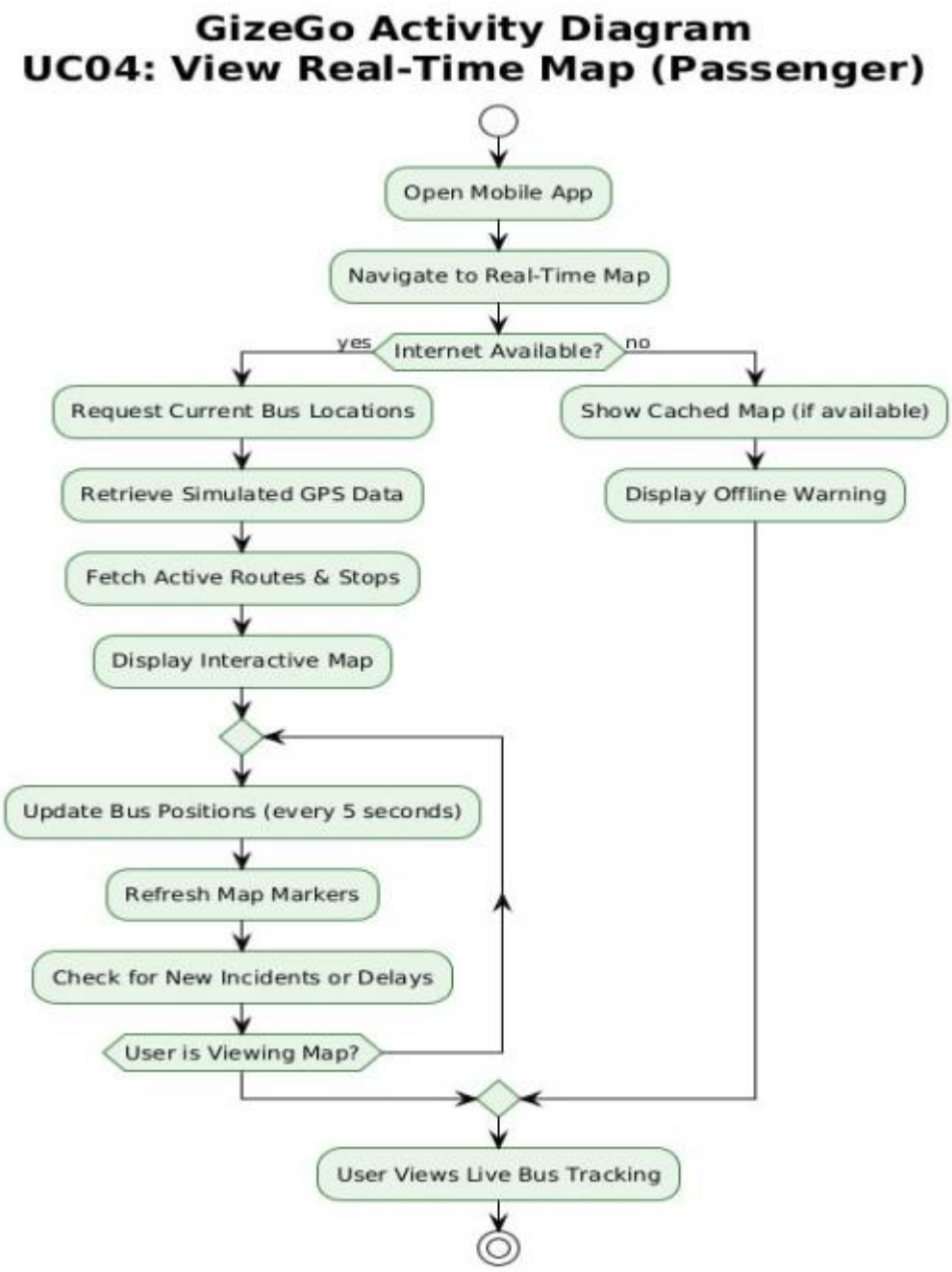


Figure 2.16 Activity diagram for view real time map

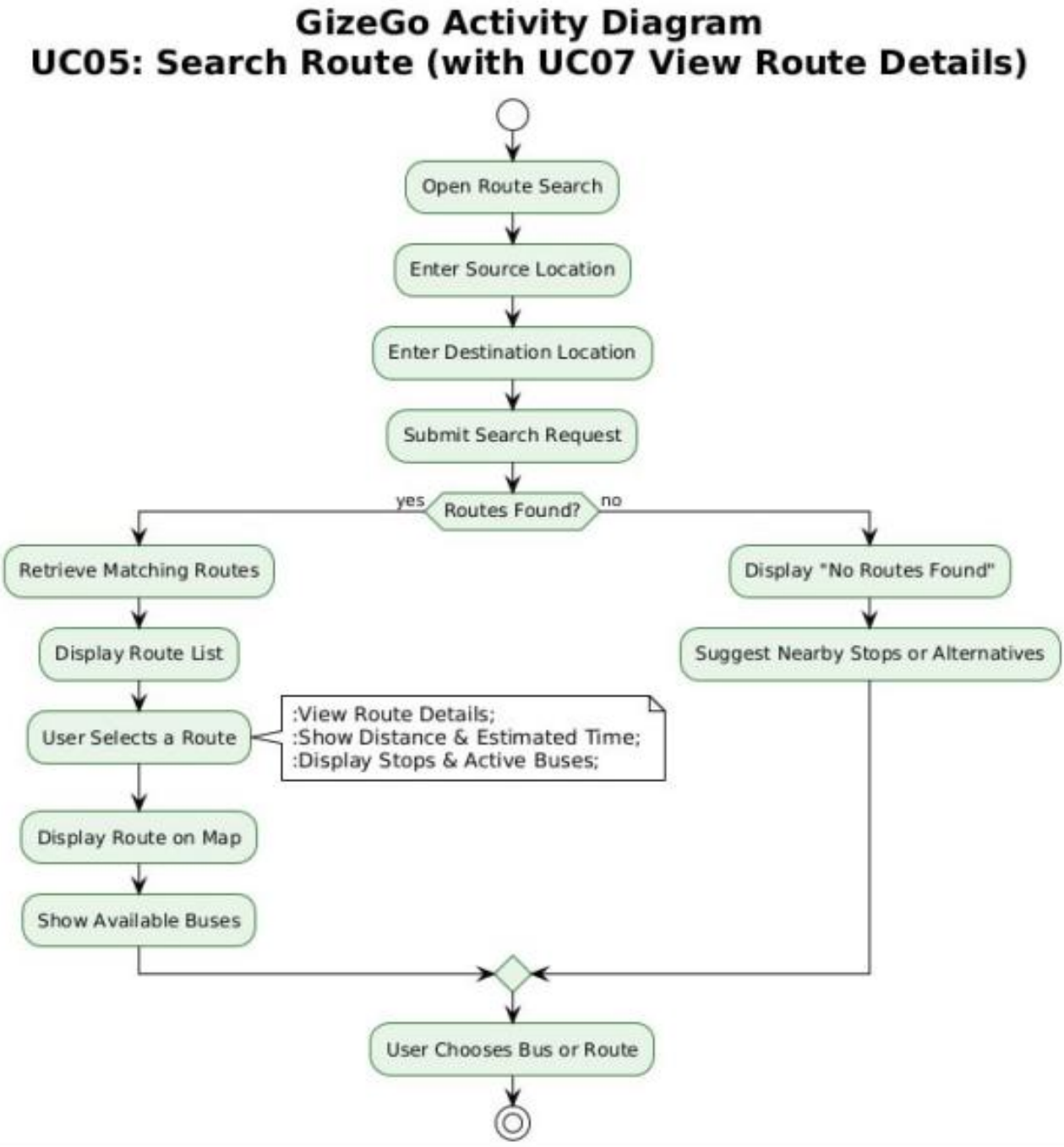


Figure 2.17 Activity diagram for Search Route



Figure 2.18 Activity diagram for view bus ETA

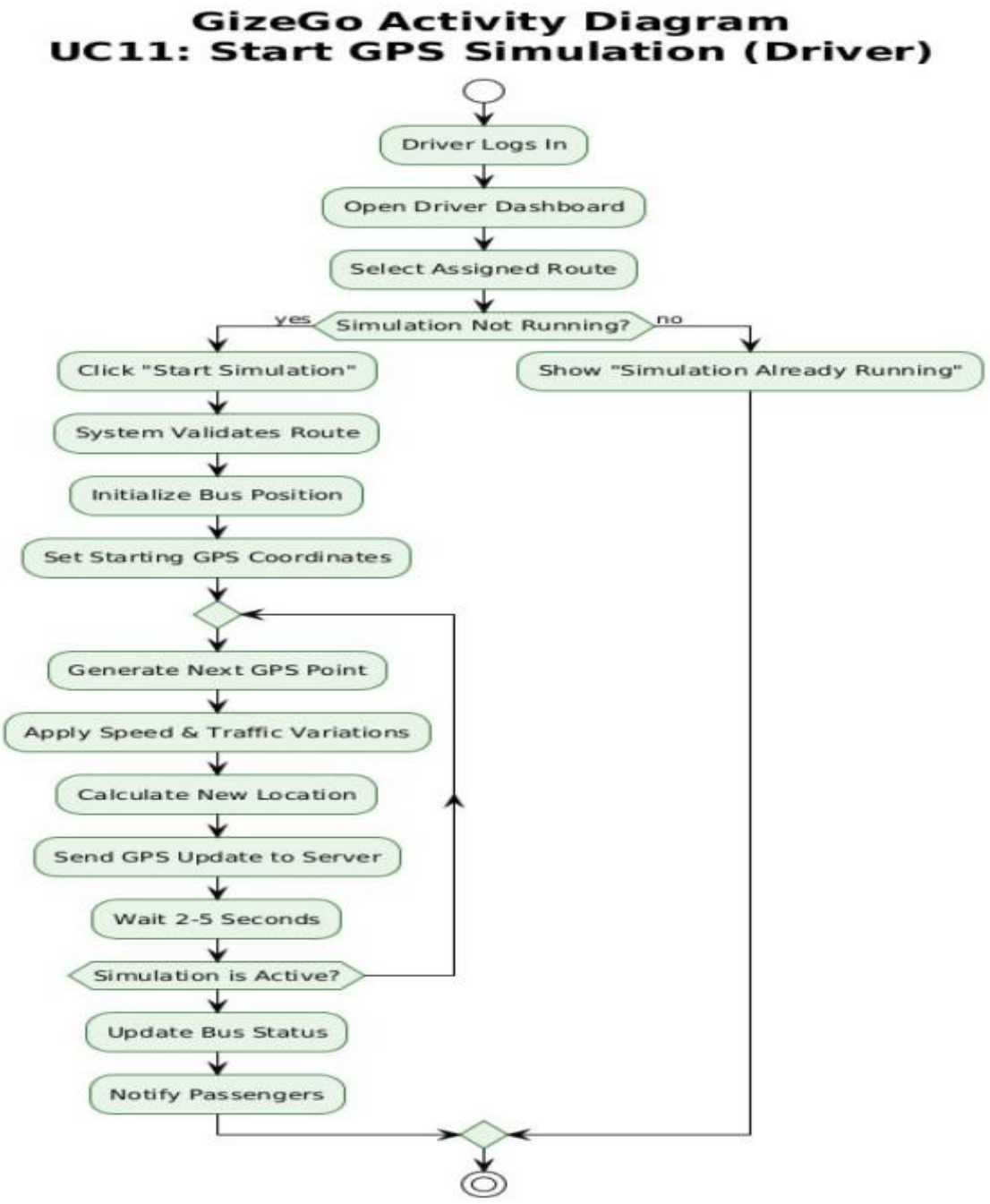


Figure 2.19 Activity diagram for Start GPS Simulation

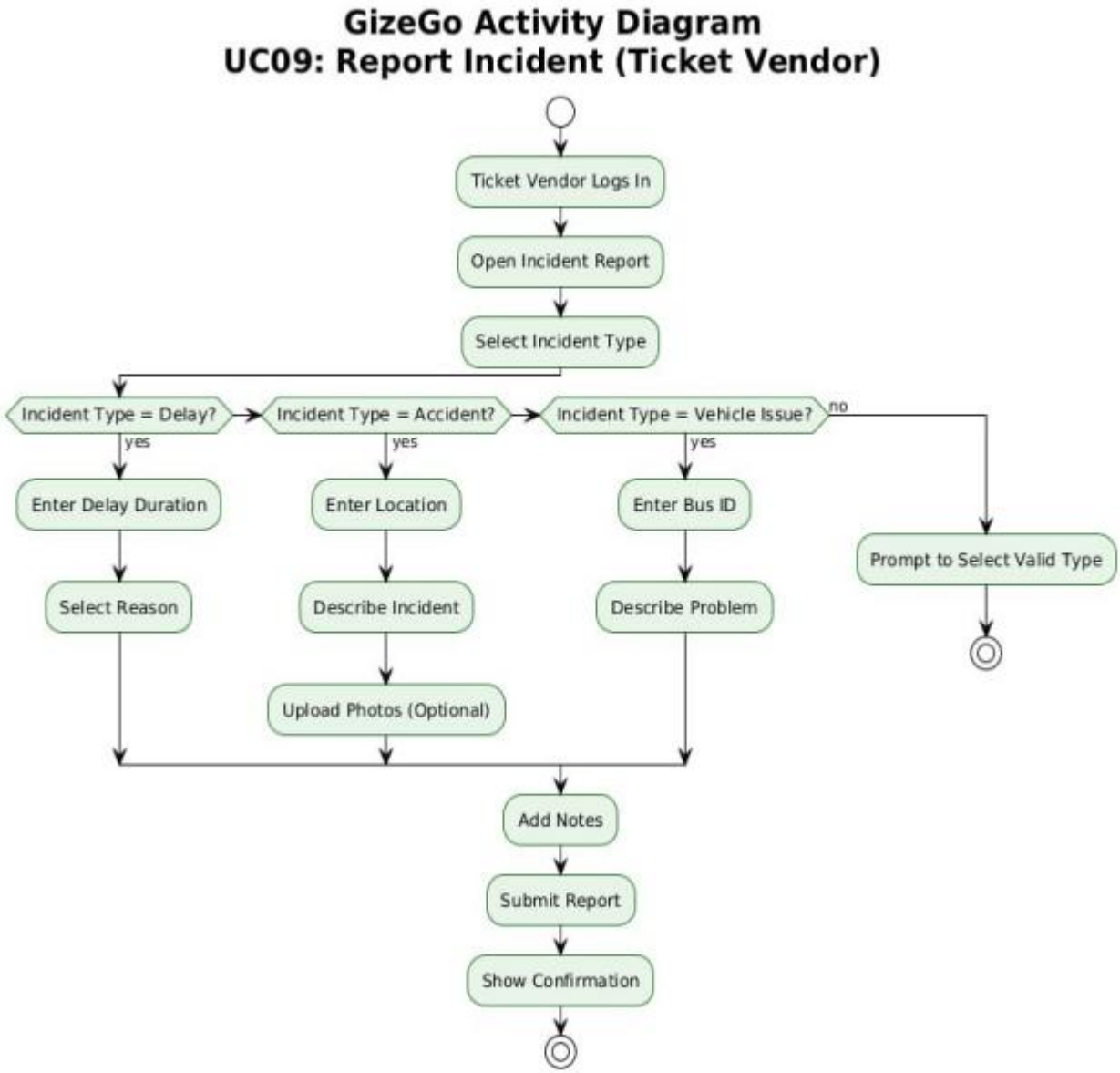


Figure 2.20 Activity diagram for Report Incident

CHAPTER THREE

SYSTEM DESIGN

3.1 Introduction

System design is the process of planning, structuring and defining the architecture of a Software System. It involves translating user requirements into a detailed blueprint that guides the implementation phase. The goal is to create a well-organized and efficient structure that meets the intended purpose while considering factors like scalability, maintainability, and performance [10].

The system design of the GizeGo Real-Time Bus Tracking System Simulation defines the technical structure used to deliver real-time bus location tracking, passenger information services, incident reporting, and administrative control. This design specifies how system components are organized, how data flows between them, and how real-time information is processed and delivered to different users. It provides a clear blueprint for implementing the proposed solution in a reliable, scalable, and maintainable manner.

GizeGo is designed as a modular, layered system that supports multiple user groups through dedicated interfaces. Passenger applications provide live bus locations, route progress, and estimated arrival times (ETAs), while driver tracking modules supply continuous location updates to the backend. An administrative dashboard enables transport operators to manage buses, routes, drivers, schedules, and incidents. The backend services handle tracking logic, ETA computation, data management, and real-time communication to ensure timely and accurate updates across all client applications.

To demonstrate system capabilities without relying on physical GPS hardware, the design incorporates a simulation-based tracking mechanism that generates realistic bus movement along actual road networks. This approach enables testing of real-time tracking, route optimization, and incident handling under controlled conditions while remaining cost-effective. Overall, the system design emphasizes real-time performance, security, scalability, and future extensibility, ensuring that GizeGo can evolve into a fully deployed urban transportation solution.

3.1.1 Purpose of the System Design

The purpose of this system design is to translate the Gizego project requirements real-time bus location tracking, passenger information services, incident handling, and administrative control into a clear technical solution that can be implemented, tested, deployed, and maintained effectively. It defines how the system will be built, how components will communicate, and how data will be collected, processed, and delivered to end users in real time.

How the design translates requirements into a technical solution

- **Real-time tracking requirement**: Each bus provides continuous GPS location updates through a driver mobile application or onboard tracking device. These updates are transmitted to a backend tracking API, stored as live telemetry data, and made available to passengers and operators with minimal delay.

- **Passenger information requirement**: Passengers access the system through a mobile application to view live bus locations, route progress, estimated arrival times (ETAs), stop information, and service alerts.

- **Operations/admin requirement**: Transport administrators use an operations dashboard to register and manage buses, routes, drivers, schedules, and monitor fleet performance (online/offline status, delays, route adherence).

- **Incident/reporting requirement**: Ticket Vendor can submit structured incident reports (breakdown, accident, heavy traffic, route disruption). Administrators review incidents, track resolution status, and broadcast alerts to affected passengers.

How system components are structured

The system is designed using a modular architecture, where each module has a clear responsibility and interacts with other modules through well-defined interfaces:

- **Client Layer**
    - Passenger mobile application
    - Driver and ticket vendor tracking application
    - Admin web dashboard

- **Backend Services Layer**
    - Core APIs (user, bus, route, schedule management)
    - Tracking and ETA processing services
    - Incident and alert management service

- **Data Layer**
    - Databases for users, routes, buses, drivers, schedules
    - Telemetry storage for bus location history and latest positions
    - Cache layer to quickly serve “latest location” and ETA queries

- **Integration Layer**
    - Map services for route visualization and map matching
    - Push notifications gateways for alerts
    - Authentication/authorization services for secure access

3.2 Proposed Software Architecture

3.2.1 Architectural Overview

The GizeGo system is designed using a **Three-Tier Architecture**, which is a common implementation of the Client-Server architectural pattern combined with a layered structure [1]. This architecture separates the system into three distinct tiers: the Presentation Tier, Application (Business Logic) Tier, and Data Tier, allowing clear separation of responsibilities and improved system maintainability.

The Presentation Tier consists of a mobile application used by passengers, administrators, drivers, and ticket vendors. It provides user interfaces for viewing real-time bus locations, checking estimated arrival times, updating bus status, and reporting incidents. The mobile application communicates with the backend through secure RESTful APIs.

The Application Tier is implemented using a Node.js backend server and contains the core business logic of the system. It handles authentication, role-based access control,

GPS simulation processing, ETA calculation, and incident management. Centralizing these functions ensures consistent behavior and secure access for all users.

The Data Tier manages persistent storage of system data, including user information, bus and route details, GPS location logs, and incident reports. Direct access to this tier is restricted to the application tier to ensure data security and integrity.

The Three-Tier Architecture was chosen because it effectively supports GizeGo's real-time processing needs, multiple user roles, and mobile-based access. It also provides scalability and flexibility for future system enhancements.

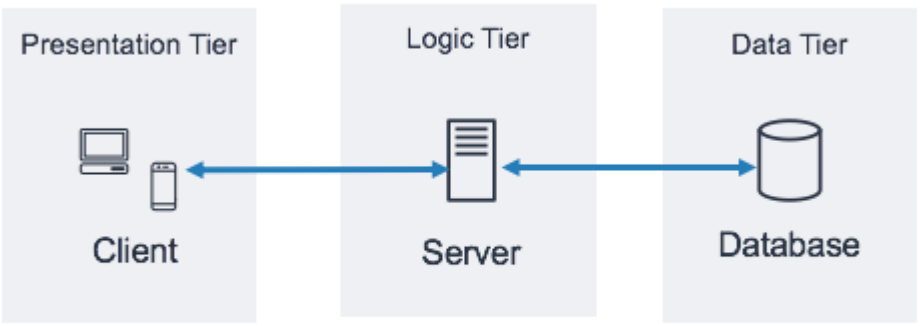


Figure 3.1 Three-Tier Architecture

3.2.2 Subsystem Decomposition

Subsystem decomposition is the process of dividing a complex system into smaller, manageable, and functionally coherent components called **subsystems**. Each subsystem performs a specific set of tasks and interacts with other subsystems through well-defined interfaces [2].

To reduce system complexity and improve maintainability, the GizeGo system is decomposed into several functional subsystems. Each subsystem is responsible for a specific set of related functionalities and interacts with other subsystems through well-defined interfaces. This decomposition follows the principles of modular design and supports the overall Three-Tier architecture.

1. User Management Subsystem

This subsystem is responsible for managing all system users, including passengers, administrators, drivers, and ticket vendors. It handles user registration, authentication, role-based authorization, and profile management. By centralizing user-related operations, the subsystem ensures secure access control and consistent enforcement of system policies.

2. Bus and Route Management Subsystem

The bus and route management subsystem manages information related to buses, routes, and stops. It allows administrators to register buses, define routes, assign drivers, and manage operational status. This subsystem provides essential data used by other subsystems, such as GPS tracking and ETA calculation.

3. GPS Simulation and Tracking Subsystem

This subsystem simulates real-time GPS data for buses operating on predefined routes. It generates location updates at fixed intervals and forwards them to the system for processing. The simulated location data is used to display real-time bus movement on the mobile application and serves as input for arrival time estimation.

4. ETA Calculation Subsystem

The ETA (Estimated Time of Arrival) calculation subsystem processes GPS location data and route information to estimate bus arrival times at upcoming stops. It applies predefined algorithms to compute accurate arrival predictions and provides this information to passengers through the mobile application.

5. Incident Management Subsystem

The incident management subsystem enables ticket vendors to report operational incidents such as delays, breakdowns, or accidents. It records incident details, updates incident status, and makes this information available to administrators for monitoring and response. This subsystem improves communication and operational awareness.

6. Notification Subsystem

This subsystem is responsible for generating and sending system notifications related to incidents, delays, or important operational updates. Notifications are delivered to relevant users, particularly administrators, to ensure timely awareness and action.

7. Data Management Subsystem

The data management subsystem handles persistent storage and retrieval of system data, including user records, bus and route information, GPS logs, and incident reports. It ensures data integrity, consistency, and controlled access by allowing only authorized subsystems to interact with the database.

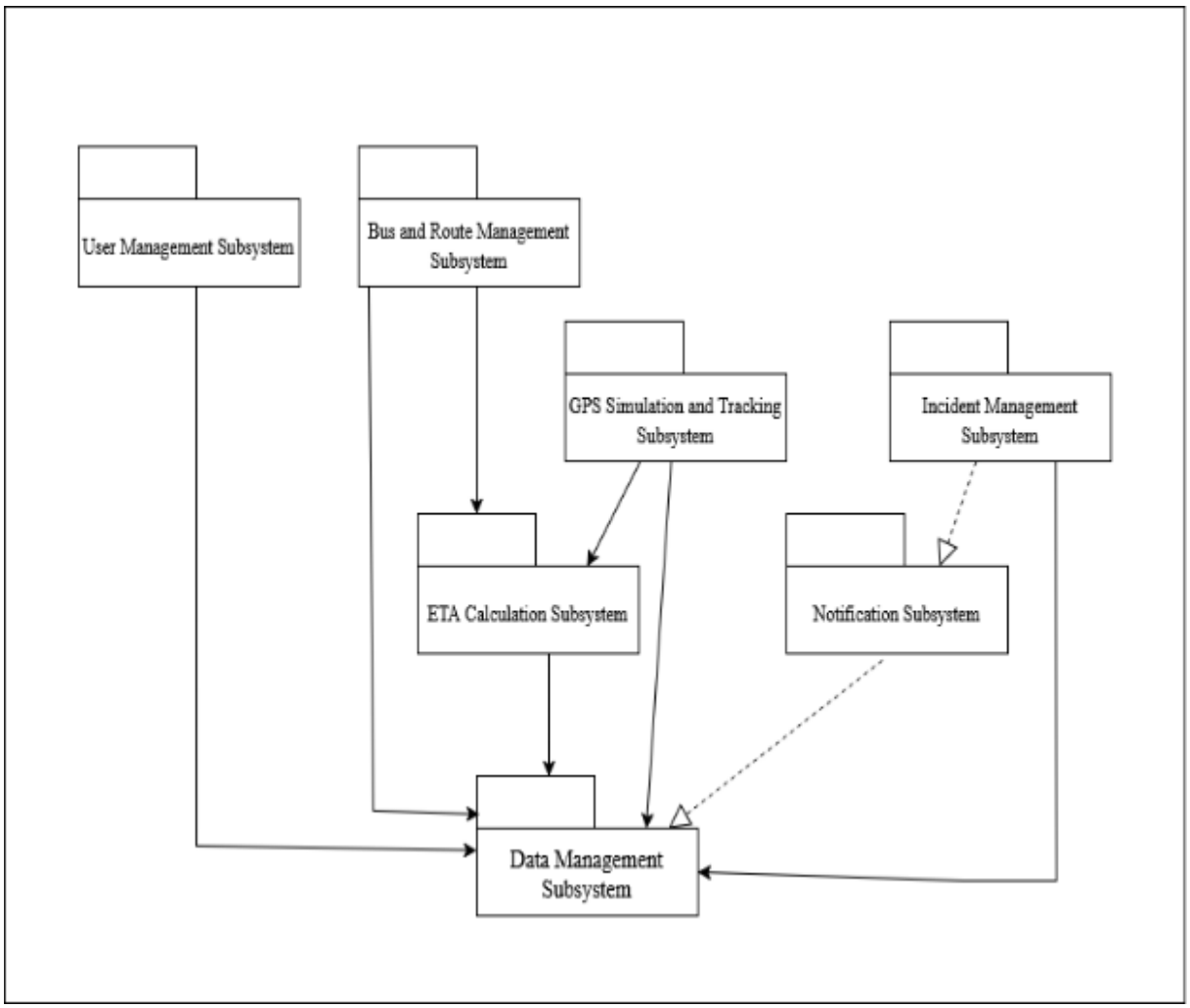


Figure 3.2 component diagram

3.2.3 Hardware / Software Mapping

Hardware/Software Mapping is the process of assigning software components (modules, subsystems, or processes) to specific hardware resources in the system[2].

The GizeGo system is designed to run efficiently by mapping each software subsystem to appropriate hardware resources. Such as:

Mobile Application (Presentation Tier):

The GizeGo mobile application is deployed on smartphones and tablets and serves as the primary interface for passengers, drivers, administrators, and ticket vendors. It provides functionalities such as route search, real-time bus tracking, incident reporting, and user interaction. The mobile device connects to the backend server through secure APIs, allowing users to perform operations while the application tier handles processing and data management.

Application Tier (Node.js Backend Server):

The application tier is deployed on a backend server and serves as the core of the GizeGo system. It processes all business logic, including user authentication, role-based access control, bus and route management, ETA calculations, GPS simulation integration, and incident management. Centralizing these functions on the backend server ensures consistent operation, security, and performance for all mobile clients.

Data Management Subsystem (Database Server):

The database subsystem is deployed on a dedicated database server. It stores critical system data, including user profiles, bus and route information, GPS logs, and incident reports. Only the backend server interacts directly with the database, ensuring secure, reliable, and persistent storage.

GPS Simulation Subsystem:

The GPS simulation subsystem runs on the backend server alongside the application tier. It generates real-time simulated GPS coordinates for buses on predefined routes and provides location updates to the backend. This allows the mobile application to display live bus locations and supports ETA calculations without relying on real GPS devices.

Notification Subsystem:

The notification subsystem is also deployed on the backend server. It handles asynchronous notifications triggered by incident reports or operational updates. When an incident occurs, this subsystem ensures that alerts are sent to relevant administrators or drivers efficiently, without affecting the core business logic.

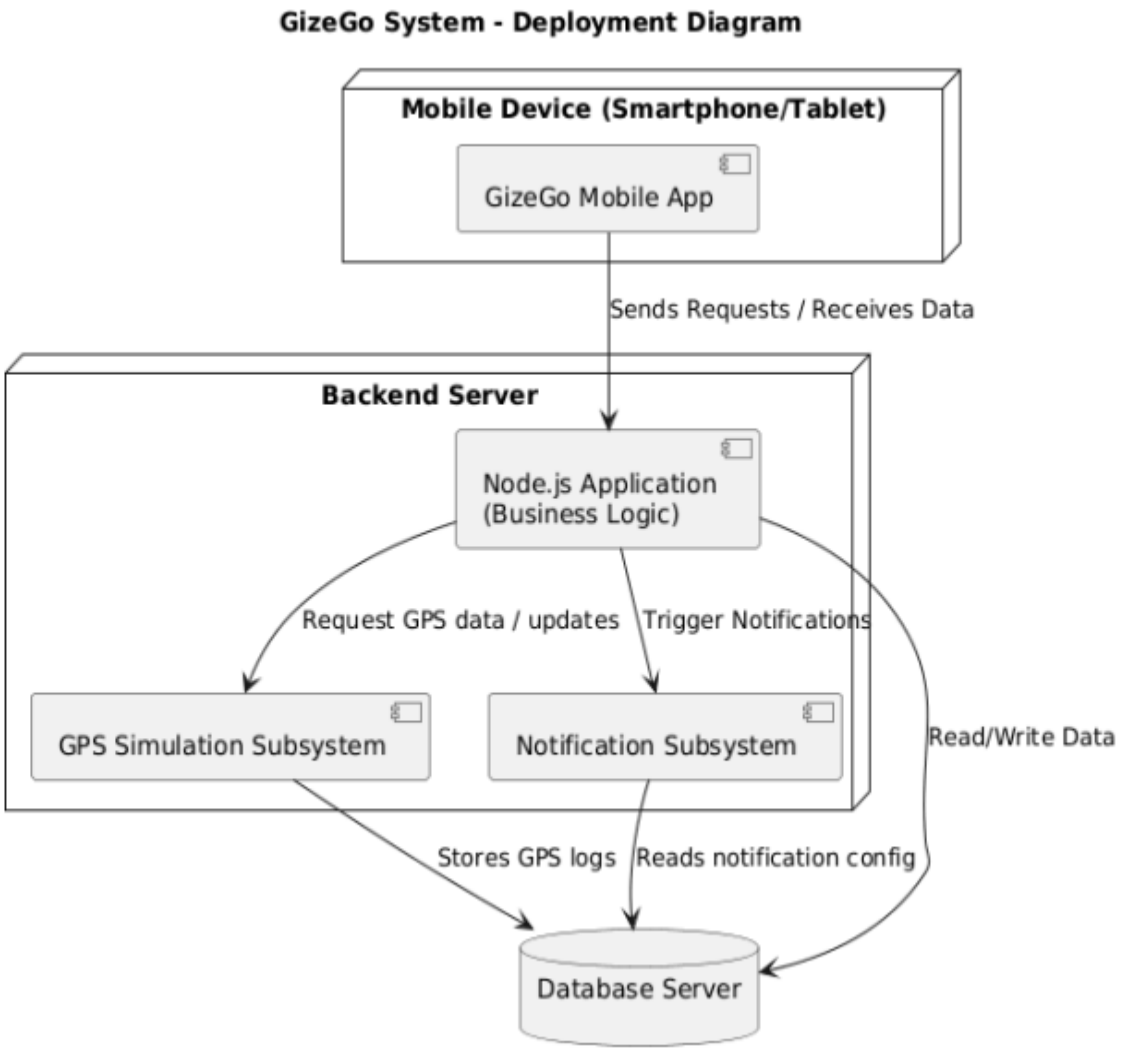


Figure 3.3 Deployment diagram

3.2.4 Persistent Data Management

The GizeGo system uses a relational database (PostgreSQL) to store all persistent data. This includes user information, bus and route details, GPS logs, and incident reports. PostgreSQL was chosen because it provides strong ACID guarantees, ensuring data consistency and integrity, which is essential for critical system operations such as tracking buses and managing user accounts.

All subsystems interact with the database through the application tier (Node.js backend server). The User Management Subsystem stores and retrieves user profiles and role

information, the Bus and Route Management Subsystem manages route and bus details, the GPS Simulation Subsystem records simulated GPS locations, and the Incident Management Subsystem logs operational incidents. Using PostgreSQL allows the system to enforce data relationships and constraints, making queries for ETA calculations, route planning, and incident reporting efficient and reliable.

Additionally, PostgreSQL supports scalability and future enhancements, allowing the system to accommodate more buses, routes, and users without significant architectural changes. Its relational structure ensures that complex queries, such as joining buses, routes, and GPS data, are handled efficiently, supporting the real-time requirements of the GizeGo system.

Table 3.1 Users table

Column name	Data type	constraints
user_id	serial	PRIMARY KEY
full_name	VARCHAR(100)	NOT NULL
phone_number/email	VARCHAR(20)	UNIQUE, NOT NULL
password	VARCHAR(255)	NOT NULL
role_id	INT	FOREIGN KEY REFERENCES roles(role_id)
status	VARCHAR(20)	DEFAULT 'active'
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Table 3.2 Roles table

Column name	Data type	constraints
role_id	SERIAL	PRIMARY KEY
role_name	VARCHAR(50)	UNIQUE, NOT NULL

Table 3.3 Buses table

Column Name	**Data Type**	**Constraints**
bus_id	SERIAL	PRIMARY KEY
plate_number	VARCHAR(20)	UNIQUE, NOT NULL
status	VARCHAR(20)	DEFAULT 'active'

Table 3.4 Routes table

Column Name	**Data Type**	**Constraints**
route_id	SERIAL	PRIMARY KEY
route_name	VARCHAR(100)	NOT NULL
start_point	VARCHAR(100)	NOT NULL end_point
end_point	VARCHAR(100)	NOT NULL

Table 3.5 Stops table

Column Name	**Data Type**	**Constraints**
stop_id	SERIAL	PRIMARY KEY
stop_name	VARCHAR(100)	NOT NULL

latitude	DECIMAL(9,6)	NOT NULL
longitude	DECIMAL(9,6)	NOT NULL

Table 3.6 Bus_Routes table (Associates buses with routes)

Column Name	**Data Type**	**Constraints**
bus_route_id	SERIAL	PRIMARY KEY
bus_id	INT	FOREIGN KEY REFERENCES buses(bus_id)
route_id	INT	FOREIGN KEY REFERENCES routes(route_id)

Table 3.7 gps_logs table

Column Name	**Data Type**	**Constraints**
gps_id	SERIAL	PRIMARY KEY
bus_id	INT	FOREIGN KEY REFERENCES buses(bus_id)
latitude	DECIMAL(9,6)	NOT NULL
longitude	DECIMAL(9,6)	NOT NULL
logged_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Table 3.8 Incidents table

Column Name	**Data Type**	**Constraints**
incident_id	SERIAL	PRIMARY KEY
bus_id	INT	FOREIGN KEY REFERENCES buses(bus_id)
reported_by	INT	FOREIGN KEY REFERENCES users(user_id)
description	TEXT	NOT NULL
incident_time	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
status	VARCHAR(20)	DEFAULT 'reported'

Table 3.9 Notification table

Column Name	**Data Type**	**Constraints**
notification_id	SERIAL	PRIMARY KEY
incident_id	INT	FOREIGN KEY REFERENCES incident _id
message	TEXT	NOT NULL
sent_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Table 3.10 ETA_predictions

Column Name	**Data Type**	**Constraints**
eta_id	SERIAL	PRIMARY KEY
bus_id	INT	FOREIGN KEY REFERENCES buses(bus_id)
stop_id	INT	FOREIGN KEY REFERENCES stops
estimated_time	TIMESTAMP	NOT NULL

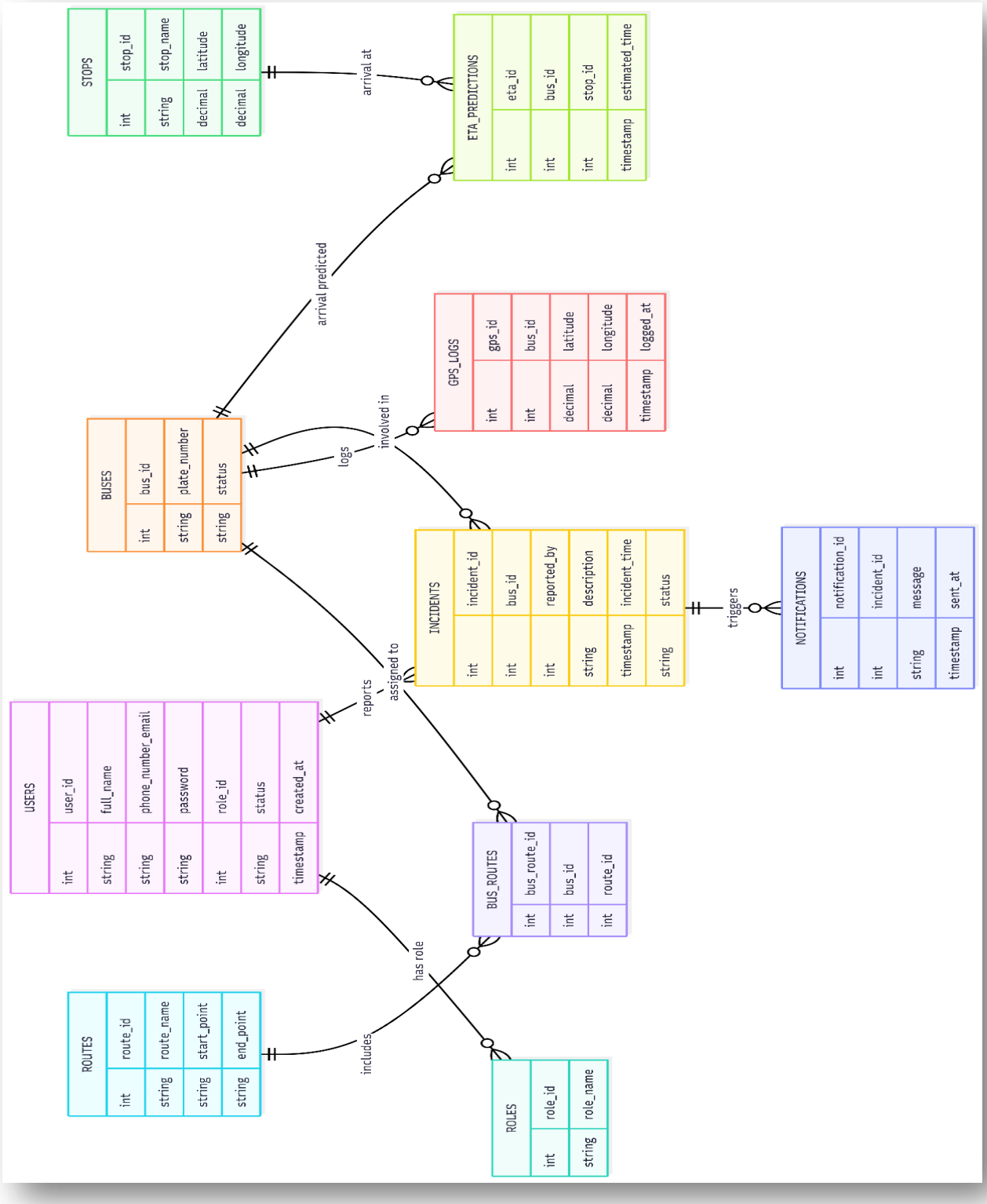


Figure 3.4 ER diagram

3.2.5 Access Control and Security

Authentication Mechanism

The GizeGo system uses a secure authentication mechanism to verify the identity of all users before granting access. Each user is required to log in with a unique phone number/ email and password. Passwords are never stored in plain text; instead, they are securely hashed using modern hashing algorithms before storage in the database. During login, the system compares the hash of the entered password with the stored hash to authenticate users. This ensures that unauthorized users cannot gain access to the system, and user credentials remain confidential.

Role Based Authorization

Once authenticated, the system enforces role-based authorization to control access to specific functionalities based on the user's role. The GizeGo system defines four primary roles: Administrator, Driver, Passenger, and Ticket Vendor. Each role is associated with a predefined set of permissions:

- Administrator**:** Can manage buses, routes, users, and view all system data.
- Driver**:** Can update bus status and view assigned routes.
- Ticket Vendor**:** Can report incidents but cannot modify routes or buses.
- Passenger: Can search routes, view estimated arrival times, and track buses in real time.

This separation ensures that users can only access resources appropriate to their responsibilities, reducing the risk of unauthorized actions.

Data Protection

The system implements multiple measures to protect sensitive data. All communication between mobile clients and the backend server is secured using HTTPS, preventing eavesdropping and man-in-the-middle attacks. Critical data, such as user passwords and personally identifiable information, is stored securely in the PostgreSQL database using encryption and hashing where appropriate. Access to the database is restricted through the backend application tier, and direct access from clients is prohibited. Regular backups are performed to prevent data loss, and the system is designed to maintain integrity even in the event of failures.

Table 3.11 Access control

Subsystem	Administrator	Driver	Ticket Vendor	Passenger
User management	Create, update, delete, view	View only	View only	View, create and update only its own profile
Bus and route management	Create, update, delete, view	View routes	View routes	View routes
GPS simulation and tracking	View bus location	View bus location	View bus location	View bus location
ETA calculation	View ETA	View ETA	View ETA	View ETA
Incident management	View incident	view incident	Report, view incident	view incident
notification	View notification	View notification	View notification	View notification
Data management	Backup, restore. Read/write data	View only	View only	View only

3.3 Detailed Class Diagram

3.3.1 Conceptual Class Diagram

A conceptual class diagram is a high-level, language-independent diagram that models the real-world concepts in a problem domain and their relationships, focusing on the business logic rather than specific code details, serving as a blueprint for understanding the system's core components and how they relate for stakeholders and analysts[3].

The conceptual class diagram represents the high-level structure of the GizeGo system without implementation-specific details. Each class corresponds directly to an entity in the ER diagram, ensuring design consistency between the conceptual and data models.

3.3.2 Description of Main Classes

1. User

The User class represents any individual interacting with the system. It stores authentication and profile information (userId, name, email, password, status) and serves as the base for all specific user types, including Passenger, Driver, Admin, and Ticket Vendor. It handles login, logout, and basic profile management.

2. Passenger

Represents commuters who access real-time bus information, such as locations, estimated arrival times (ETAs), and route information. Passenger-specific operations include viewing buses, searching routes, and receiving notifications.

3. Driver

Represents bus drivers responsible for initiating and controlling GPS simulation for bus movement. Driver-specific operations include starting and stopping bus simulations.

4. Admin

Admin represents system administrators who oversee system operations, manage users and routes, and monitor fleet and performance. They can view analytics and incident reports.

5. Ticket Vendor

Ticket Vendors represent on-site staff at bus stations or on buses. They are responsible for reporting operational issues, delays, or incidents to the system, enabling administrators and operators to respond efficiently.

6. Bus

The Bus class models public transport vehicles in the system. It stores bus-specific information such as bus ID, plate number, capacity, and current operational status. Each bus is assigned to a route and continuously updates its status.

7. Route

The Route class represents predefined transport routes within the system. It contains route identification data, route name, and total distance. Routes are composed of multiple stops and are used for route optimization.

8. Stop

The Stop class represents individual bus stops along a route. Each stop contains geographical coordinates and a stop name, which are essential for route visualization and ETA calculations.

9. GPS Simulator

The GPS Simulator class is a control class responsible for generating simulated GPS coordinates. It updates bus positions at fixed intervals based on route data and simulation speed, eliminating the need for real GPS hardware.

10. Bus Location

The Bus Location class stores real-time geographical information of a bus, including latitude, longitude, and timestamp. It is updated by the GPS simulator and used by ETA and map services.

11. Traffic Model

The Traffic Model class simulates traffic conditions by generating traffic weights. These weights influence route optimization and ETA calculation, allowing the system to reflect realistic congestion scenarios.

12. Route Optimizer

The Route Optimizer class computes optimal and alternative routes using shortest-path algorithms such as A*. It considers both distance and simulated traffic conditions to determine efficient routes.

13. ETA Service

The ETA Service class calculates estimated arrival times for buses at stops. It uses bus location data, route information, and average speed to provide accurate arrival predictions to passengers.

14. Notification

The Notification class handles system-generated alerts and messages. It is responsible for notifying users about delays, route changes, or other important system events.

15. Map Service

The Map Service class acts as the interface between the system and external map services such as OpenStreetMap and Leaflet. It is responsible for rendering maps and displaying real-time bus markers.

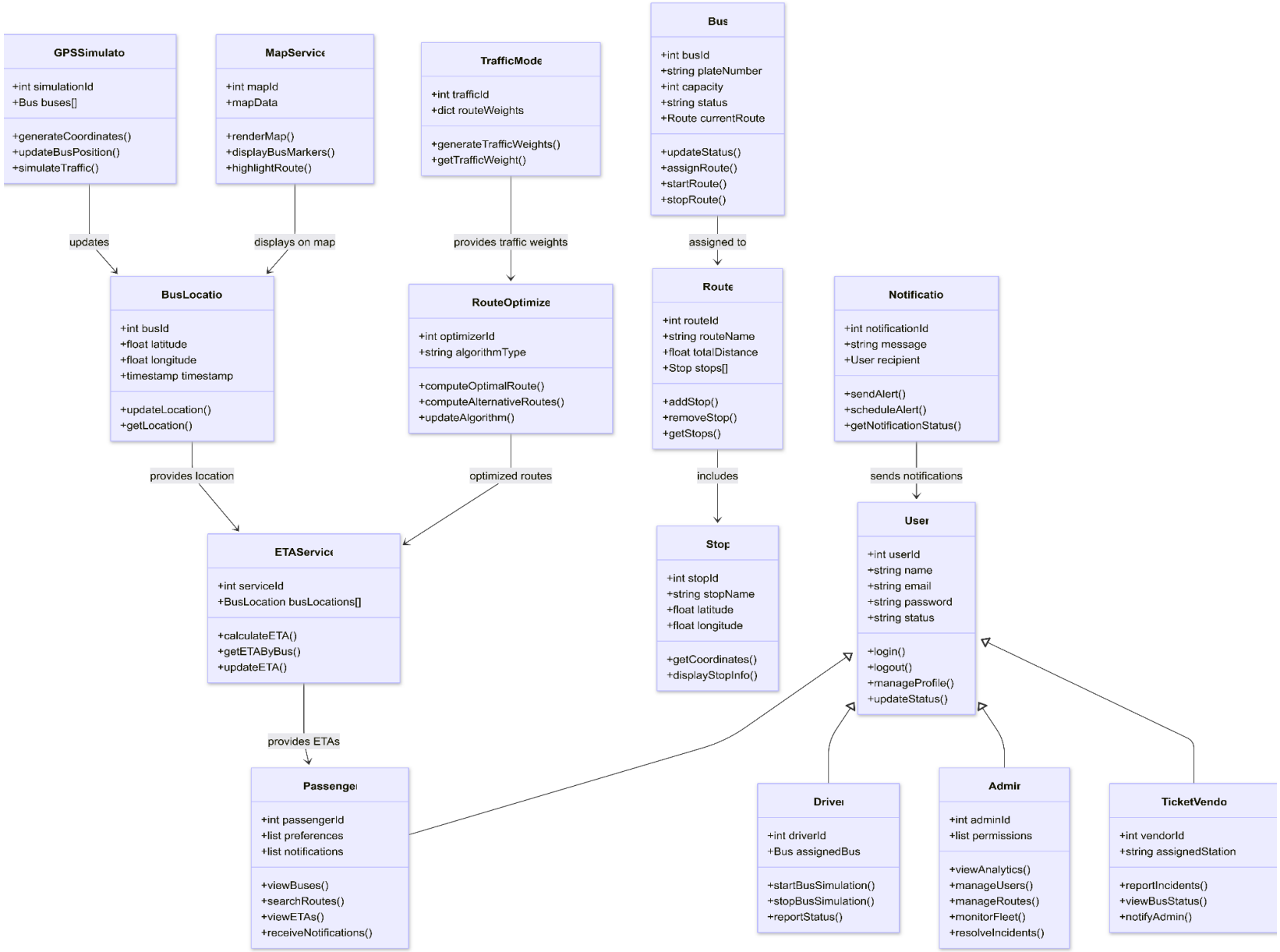

GPSSimulato
+int simulationId
+Bus buses[]
+generateCoordinates()
+updateBusPosition()
+simulateTraffic()
MapServic
+int mapId
+mapData
+renderMap()
+displayBusMarkers()
+highlightRoute()
TrafficMode
+int trafficId
+dict routeWeights
+generateTrafficWeights()
+getTrafficWeight()
Bus
+int busId
+string plateNumber
+int capacity
+string status
+Route currentRoute
+updateStatus()
+assignRoute()
+startRoute()
+stopRoute()
updates
displays on map
provides traffic weights
assigned to
BusLocatio
+int busId
+float latitude
+float longitude
+timestamp timestamp
+updateLocation()
+getLocation()
RouteOptimize
+int optimizerId
+string algorithmType
+computeOptimalRoute()
+computeAlternativeRoutes()
+updateAlgorithm()
Route
+int routeId
+string routeName
+float totalDistance
+Stop stops[]
+addStop()
+removeStop()
+getStops()
Notificatio
+int notificationId
+string message
+User recipient
+sendAlert()
+scheduleAlert()
+getNotificationStatus()
provides location
optimized routes
includes
sends notifications
ETAServic
+int serviceId
+BusLocation busLocations[]
+calculateETA()
+getETAByBus()
+updateETA()
Stop
+int stopId
+string stopName
+float latitude
+float longitude
+getCoordinates()
+displayStopInfo()
User
+int userId
+string name
+string email
+string password
+string status
+login()
+logout()
+manageProfile()
+updateStatus()
provides ETAs
Passenge
+int passengerId
+list preferences
+list notifications
+viewBuses()
+searchRoutes()
+viewETAs()
+receiveNotifications()
Drive
+int driverId
+Bus assignedBus
+startBusSimulation()
+stopBusSimulation()
+reportStatus()
Admi
+int adminId
+list permissions
+viewAnalytics()
+manageUsers()
+manageRoutes()
+monitorFleet()
+resolveIncidents()
TicketVendo
+int vendorId
+string assignedStation
+reportIncidents()
+viewBusStatus()
+notifyAdmin()

Figure 3.5 Class Diagram

3.4 Package Diagram

A package diagram is a type of structural diagram in UML (Unified Modeling Language) that organizes and groups related classes and components into packages. It visually represents the dependencies and relationships between these packages, helping to illustrate how different parts of a system interact with each other [11].

This section presents the package-level architectural design of the **GizeGo system**. The package diagram illustrates how system components are grouped into cohesive packages and how these packages interact with one another. The objective of this design is to ensure modularity, separation of concerns, scalability, and ease of maintenance.

The package organization follows a **layered architecture**, where each layer has clearly defined responsibilities and controlled dependencies. This structure supports the real-time, simulation-based nature of the GizeGo system and aligns with the system requirements and class design presented in previous sections.

3.4.1 Overview of Package Organization

The GizeGo system is structured into four primary architectural layers:

1. Presentation Layer
2. Business Logic Layer
3. Data Access Layer
4. Integration and Utilities Layer

Each layer encapsulates a set of related functionalities and interacts only with adjacent layers, ensuring loose coupling and high cohesion.

3.4.2 Presentation Layer

The Presentation Layer is responsible for all user interaction and visualization tasks. It handles input collection, interface rendering, and the display of real-time system data. This layer does not contain business rules or direct database access.

3.4.2.1 Mobile Application Package

The Mobile Application package provides interfaces for passengers and drivers. It enables users to interact with the system through map-based views, route exploration, ETA visualization, and notification reception. The package communicates with backend services to retrieve real-time bus locations and system updates while maintaining bilingual support.

Key responsibilities of this package include:

- Rendering user-friendly mobile interfaces
- Displaying simulated real-time bus movement on maps
- Presenting estimated arrival times for selected stops
- Allowing users to search and view routes
- Delivering system and incident notifications
- Supporting multilingual interaction

3.4.2.2 Dashboard Package

The Dashboard package supports administrative and operational users. It provides tools for monitoring fleet operations, managing routes and users, and visualizing system analytics. This package serves as the control and monitoring interface of the GizeGo system.

Key responsibilities of this package include:

- Monitoring bus movement and system status
- Managing transport routes and operational data
- Handling user administration tasks
- Displaying analytical reports and performance metrics

3.4.3 Business Logic Layer

The Business Logic Layer implements the core functionality of the GizeGo system. It coordinates system workflows, enforces business rules, and processes real-time and historical data. This layer acts as the intermediary between user interfaces and persistent storage.

3.4.3.1 Route Management Package

The Route Management package handles all operations related to transport routes and stops. It is responsible for maintaining route structures, validating route data, and computing optimal paths. The package uses OpenStreetMap data to ensure realistic route representation within the system.

Its responsibilities include:

- Managing route and stop information
- Validating route consistency and structure
- Computing optimized paths using routing algorithms
- Supporting route search and navigation logic

3.4.3.2 Tracking Service Package

The Tracking Service package manages the simulation and updating of bus locations. Since the system does not rely on physical GPS hardware, this package generates virtual GPS coordinates and simulates bus movement along predefined routes. It ensures continuous and realistic tracking behavior.

Its responsibilities include:

- Generating simulated GPS data
- Updating bus locations at fixed intervals
- Simulating traffic delays and movement variation
- Broadcasting location updates in real time

3.4.3.3 ETA Computation Package

The ETA Computation package calculates estimated arrival times for buses at various stops. It processes distance, speed, and traffic simulation data to produce dynamic ETA values that change in response to bus movement and simulated conditions.

Its responsibilities include:

- Calculating estimated arrival times
- Updating ETA values dynamically
- Analyzing speed and traffic conditions

- Providing ETA information to the presentation component.

3.4.3.4 Notification Service Package

The Notification Service package manages system-generated and user-targeted notifications. It processes incident reports, generates alerts, and delivers notifications to users in a timely manner. This package ensures that passengers and operators are informed about delays, route changes, and system events.

Its responsibilities include:

- Handling incident and alert generation
- Managing notification delivery
- Supporting real-time user alerts
- Tracking notification status

3.4.3.5 Authentication and Authorization Package

This package is responsible for managing user identity and access control. It ensures that only authorized users can access specific system features and enforces role-based permissions across the system.

Its responsibilities include:

- User authentication
- Role-based access control
- Session and token management
- Credential validation and security enforcement

3.4.3.6 Analytics Package

The Analytics package processes historical and real-time system data to generate insights into operational performance. It supports monitoring, reporting, and decision-making by analyzing tracking data, incidents, and system usage patterns.

Its responsibilities include:

- Aggregating operational data
- Analyzing system performance

- Generating analytical reports
- Preparing data for dashboard visualization

3.4.4 Data Access Layer

The Data Access Layer manages persistent data storage and retrieval. It isolates the Business Logic Layer from database-specific concerns and ensures consistent data access across the system.

Key responsibilities of this layer include:

- Executing data storage and retrieval operations
- Managing database transactions
- Ensuring data consistency and integrity
- Providing repository-level abstraction

3.4.5 Integration and Utilities Layer

This layer contains supporting services and shared utilities used across the entire system. It includes integration with external services and common system-wide utilities.

3.4.5.1 External Services Package

The External Services package enables interaction with third-party systems and external data sources. It primarily supports map data integration and real-time communication.

Its responsibilities include:

- Integrating OpenStreetMap data
- Supporting real-time data transmission
- Providing external notification mechanisms

3.4.5.2 Utilities Package

The Utilities package provides reusable services and helper functions that support all layers of the system.

Its responsibilities include:

- Logging and error handling
- Input validation
- Language translation support
- Centralized configuration management

3.4.6 Package Dependencies

The GizeGo system enforces a controlled dependency structure to maintain architectural integrity:

- The Presentation Layer depends on the Business Logic Layer
- The Business Logic Layer depends on the Data Access Layer
- The Business Logic Layer depends on the Integration and Utilities Layer
- Lower layers do not depend on higher layers

This dependency model ensures loose coupling, improved testability, and long-term maintainability.

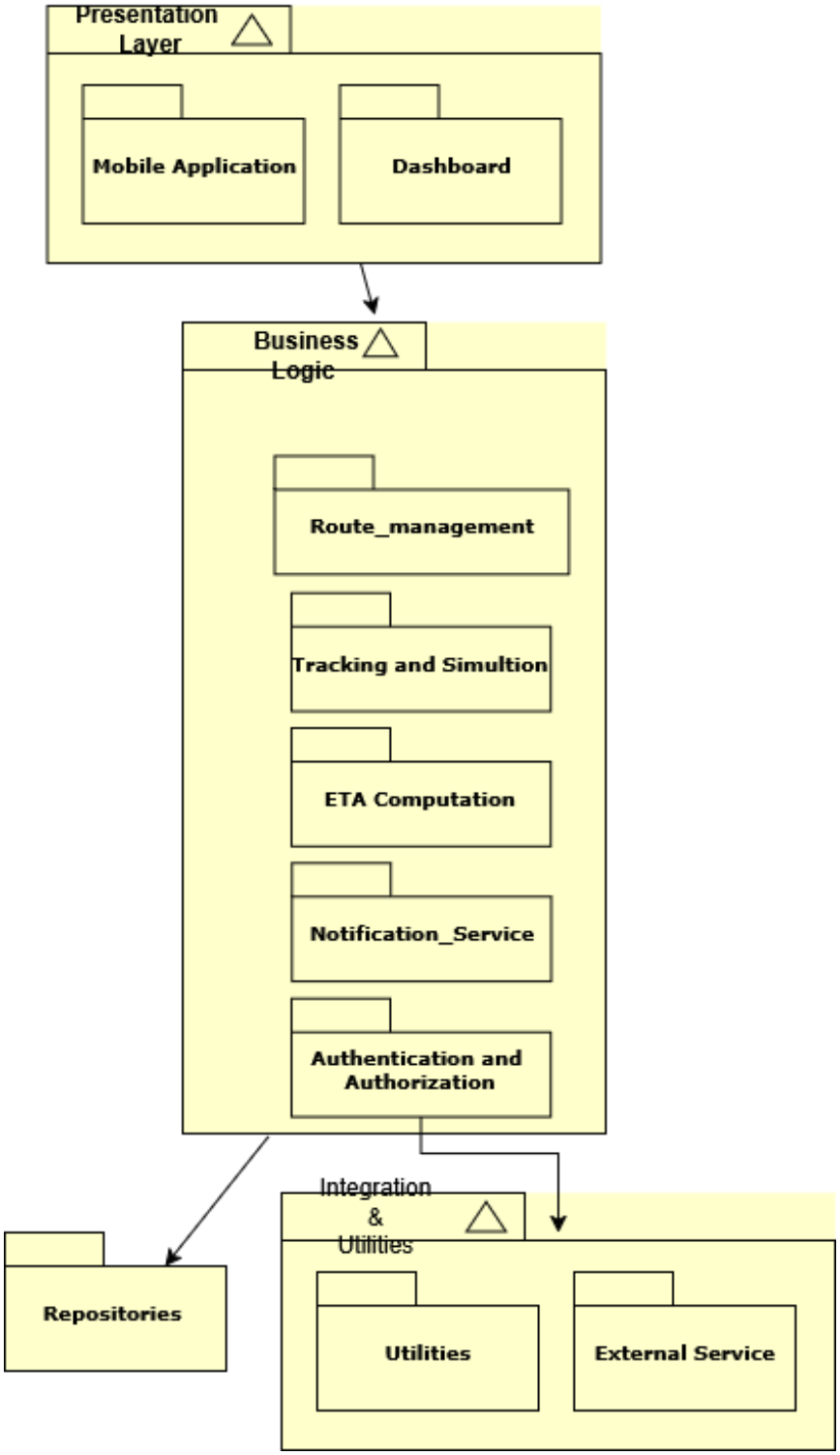


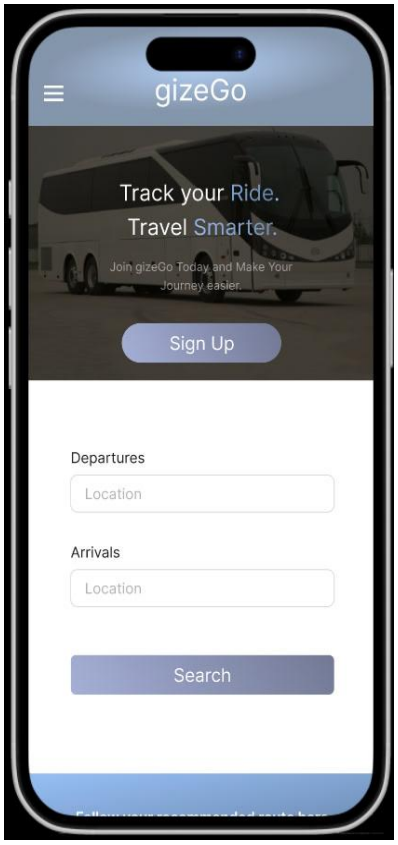
Figure 3.6 Package Diagram

3.5 UI Design

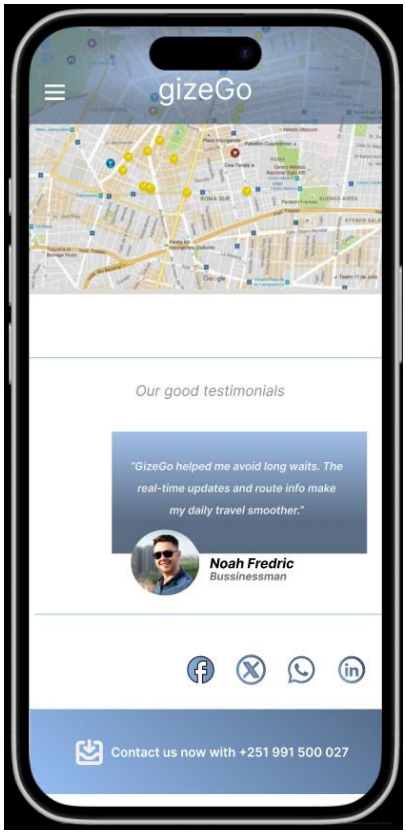
The user interface design of the GizeGo system focuses on simplicity, clarity, and real-time usability to ensure that different user groups can interact with the system effectively. Since the primary purpose of the system is to provide real-time bus tracking and transportation information, the UI is designed to present location data, routes, and alerts in a clear and intuitive manner, minimizing user effort and cognitive load.

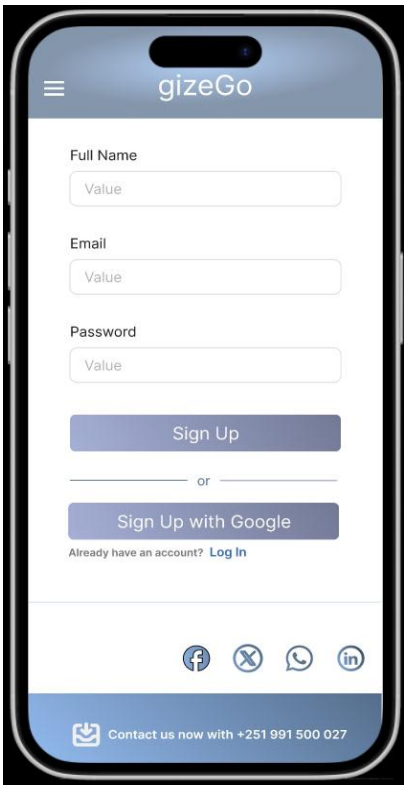
The system includes separate user interfaces for passengers and transport administrators. The passenger interface emphasizes an interactive map view that displays live bus locations, routes, stops, and estimated arrival times (ETAs). Key information such as delays, service alerts, and route progress is visually highlighted to allow users to quickly understand the current transport status. The administrative interface focuses on monitoring and management tasks, providing dashboards, tables, and charts to visualize fleet status, route performance, incident reports, and system activity.

All user interface screens, layouts, and navigation flows were designed and prototyped using Figma. Figma was used to create wireframes and high-fidelity mockups that define screen structure, component placement, color usage, and user interactions before implementation. This design-first approach improved consistency across screens, supported usability evaluation, and served as a clear reference during frontend development using React Native.


gizeGo
Track your Ride.
Travel Smarter.
Join gizeGo Today and Make Your Journey easier.
Sign Up
Departures
Location
Arrivals
Location
Search


gizeGo
Follow your recommended route here


gizeGo
Our good testimonials
"GizeGo helped me avoid long waits. The real-time updates and route info make my daily travel smoother."
Noah Fredric
Bussinessman
Contact us now with +251 991 500 027


gizeGo
Full Name
Value
Email
Value
Password
Value
Sign Up
or
Sign Up with Google
Already have an account? Log In
Contact us now with +251 991 500 027

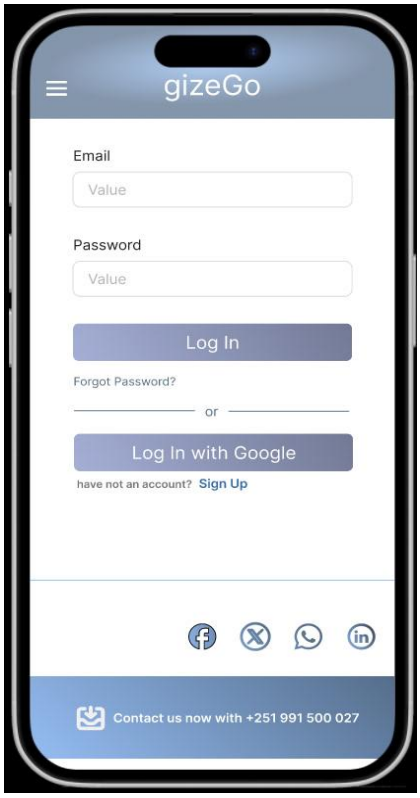


Figure 3.7 UI design

3.6 Design Constraints and Assumptions

3.6.1 Design Assumptions

Design assumptions are conditions or situations that are taken for granted during the design process.

These assumptions help simplify design decisions but must be validated or monitored during implementation and deployment.

3.6.1.1 User Assumptions

Smartphone and Internet Access:

- It is assumed that target users have access to smartphones capable of running the mobile application.
- Users are assumed to have basic internet connectivity, even if intermittent or slow. - Users are familiar with basic mobile application usage and navigation.

- Validation: User surveys and pilot testing can validate these assumptions. Fallback mechanisms should be considered for users with limited access.

User Technical Literacy:

- Users are assumed to understand basic mobile app operations (tapping, scrolling, searching).
- Users can navigate bilingual interfaces and switch languages as needed.
- Validation: Usability testing and user training scenarios can confirm these assumptions.

3.6.1.2 Data and Mapping Assumptions

OpenStreetMap Data Quality:

- OpenStreetMap provides sufficiently accurate route and road network data for major areas of Addis Ababa.
- OSM data includes adequate coverage of bus routes and stops used in the system.
- Map data updates are available when needed (though not necessarily real-time).
- Validation: Route validation against actual bus routes can verify OSM data accuracy. Manual corrections may be required for discrepancies.

Simulated GPS Accuracy:

- Simulated GPS data can reasonably represent real-world bus movement behavior. -
- Simulation-generated coordinates produce realistic bus trajectories and speed patterns. -
- Traffic condition simulation accurately reflects congestion effects on bus movement.
- Validation: Comparison with actual GPS data (if available) or expert review can validate simulation realism.

3.6.1.3 System Operation Assumptions

Traffic Condition Simulation:

- Traffic conditions can be approximated using simulated traffic weights and time-based variations.
- Simulated traffic patterns reflect real-world peak hours, congestion zones, and delay scenarios.
- Validation: Traffic simulation can be calibrated against actual traffic data or expert knowledge of Addis Ababa traffic patterns.

Route Optimization:

- The A pathfinding algorithm produces acceptable route optimization results for the intended use case.
- Optimized routes computed by the system are practical and feasible for actual bus operations.
- Validation:* Route optimization results can be reviewed by transport operators and compared with manual route planning.

3.6.1.4 Performance Assumptions

Response Time Expectations:

- Users accept response times within specified quality requirements (e.g., < 3 seconds for registration, < 2 seconds for route search). -
- Network latency does not exceed reasonable thresholds for real-time updates.
- Validation: Performance testing under various network conditions can validate these assumptions.

System Load:

- The simulated system load (10-20 buses, limited concurrent users) remains within the capacity of available infrastructure.
- Databases can handle GPS update frequency without performance degradation.
- Validation: Load testing with simulated concurrent users and GPS updates can validate system capacity.

3.6.1.5 Deployment Assumptions

Hosting Environment:

- Free-tier hosting services (or local hosting) provide sufficient resources for development and demonstration.
- Server infrastructure remains stable and accessible during system operation.
- Validation: Monitoring and resource usage tracking can validate hosting assumptions. Scalability plans should account for resource limitations.

Demonstration and Academic Use:

- The system is primarily used for demonstration and academic evaluation purposes.
- Production-level reliability and uptime requirements are lower than commercial deployment.
- Validation: This assumption aligns with project scope and can be confirmed through project objectives.

3.6.1.6 Future Integration Assumptions

Real GPS Hardware Integration: - The system architecture supports future integration with real GPS devices when available.

- Simulated GPS subsystems can be replaced or extended with real GPS data feeds.
- Validation: Architecture review and modular design can ensure future extensibility.

Scalability for Expansion:

- The system can be scaled to support additional routes, buses, and users in future phases.
- Database and server architecture accommodate growth without fundamental design.
- Validation: Architecture design patterns (e.g., horizontal scaling, database sharding) can Several risks arise from the constraints and assumptions outlined above. Mitigation strategies include:

1. Network Reliability Risks: Implement offline data caching, optimistic UI updates, and reconnection mechanisms.

2. OSM Data Accuracy Risks: Provide manual route correction tools and validate routes with transport operators.

3. Performance Risks: Conduct load testing, optimize database queries, and implement performance monitoring.

4. User Adoption Risks: Provide user training materials, conduct usability testing, and maintain bilingual support.

5. Simulation Accuracy Risks: Validate simulation parameters with transport experts and refine traffic models iteratively.

6. These constraints and assumptions shape the GizeGo system design, ensuring that the solution remains feasible, practical, and aligned with available resources and project objectives.

Conclusion

The GizeGo project successfully presents a simulation-based prototype for real-time public transportation tracking and route optimization tailored to the context of Addis Ababa. By addressing key challenges such as the lack of real-time bus information, unpredictable schedules, and inefficient route management, the system demonstrates how modern software technologies can enhance urban mobility even in environments with limited digital infrastructure.

Through the integration of simulated GPS data, OpenStreetMap resources, and routing algorithms, GizeGo provides commuters with timely and reliable information while offering operators analytical insights to support better decision-making. The use of open-source tools and a scalable system architecture ensures that the solution remains cost-effective, flexible, and adaptable for future real-world deployment.

Overall, this project contributes both practically and academically by showcasing the feasibility of intelligent transport systems in developing cities. GizeGo aligns with Ethiopia's digital transformation goals and lays a strong foundation for future enhancements, such as the integration of real GPS devices, mobile applications, predictive analytics, and collaboration with public transportation authorities. The outcomes of this project highlight the potential of technology-driven solutions to improve public transportation efficiency, reliability, and user experience.

Reference

[1] S. T. Demirel and R. Das, “Software requirement analysis: Research challenges and technical approaches,” in *2018 6th International Symposium on Digital Forensic and Security (ISDFS)*, Mar. 2018, pp. 1–6. doi: 10.1109/ISDFS.2018.8355322.

[2] “Full article: Gaps in failure management of public transport supply in Addis Ababa: a comprehensive assessment.” Accessed: Dec. 27, 2025. [Online]. Available: https://www.tandfonline.com/doi/full/10.1080/21650020.2025.2493104

[3] “Unified Modeling Language (UML) Diagrams,” GeeksforGeeks. Accessed: Dec. 27, 2025. [Online].Available:https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction/

[4] Moovit, "How Moovit works," Moovit Media Kit, 2023. [Online]. Available: https://moovit.com/. [Accessed: Oct. 15, 2024].

[5] M. Abebe and T. Dawit, "Prospects and Challenges of Digital Bus System Implementation: The Case of Addis Ababa City," Ethiopian Journal of Engineering and Technology, vol. 3, no. 1, pp. 45-58, Jun. 2020.

[6] S. Williams, J. White, P. Waiganjo, D. Orwa, and C. R. deSouza, "The Digital Matatu Project: Using Mobile Data to Reframe the Impact of Bus Rapid Transit in Nairobi," Journal of Transport Geography, vol. 49, pp. 5-9, 2015. doi: 10.1016/j.jtrangeo.2015.10.004.

[7] M. Haklay and P. Weber, "OpenStreetMap: User-Generated Street Maps," IEEE Pervasive Computing, vol. 7, no. 4, pp. 12-18, Oct. 2008. doi: 10.1109/MPRV.2008.8

[8] *OpenStreetMap Documentation, Accessed Nov 2025.*

[9] Russell, S., & Norvig, P. (2010). Artificial Intelligence: A Modern Approach (3rd ed.)

[10] “System Design Introduction - LLD & HLD,” GeeksforGeeks. Accessed: Jan. 27, 2026. [Online].Available:https://www.geeksforgeeks.org/system-design/getting-started-with-system-design/

[11].“Package Diagram – Unified Modeling Language (UML),” GeeksforGeeks. Accessed: Jan. 27,2026.[Online].Available:https://www.geeksforgeeks.org/system-design/package-diagram-introduction-elements-use-cases-and-benefits/